Residual-Slack Compatible Descendant Completeness for Locked NAND

A machine-checkable certificate report for $P = NP$ over the locked NAND residual-slack package

Residual-band exact minimization, locked NAND SAT embedding, traceable normalization, saturated support calculus, projection-defect cellization, hereditary and budget closure, selector realization, and generated proof-package checking

Complete machine-checkable proof report.

Accepted proof-report boundary.

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept} \quad \Longrightarrow \quad P = NP.$$

This paper records the completed finite-certificate route and its executable release seal. The repository emits an accepted `FinalPNPProofReport0` whose theorem field is $P = NP$, whose antecedent is $\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept}$, and whose final acceptance/replay, final certificate, release gate, release audit, and proof report records all accept. The hardened validation log for the sealed repository state reports 1043 passing tests, no failures, no cancellations, and an accepted final proof-report summary and full check record. Release custody separates the source/checker revision from the durable artefact-bearing release: source tag `final-pnp-proof-report-hardened-8b45da4`; sealed artefact tag `final-pnp-proof-report-artifacts-hardened-8b45da4-sealed`. The materialized proof-report artefact bundle is committed under `proof-artifacts/final-pnp-proof-report-hardened-8b45da4/`; its manifest is `proof-artifacts/final-pnp-proof-report-hardened-8b45da4/release-seal.json`, and non-self file-level checksums are recorded in `proof-artifacts/final-pnp-proof-report-hardened-8b45da4/SHA256SUMS`. If repository access is restricted, request access from pnp@keaton.com.au.

This paper states the final theorem under the explicit checker trust model and reproducibility protocol recorded in Section 23.

Contents

1 Executive status and claim boundary

1.1 Main message

This paper proves

$$P = NP$$

by reducing SAT to exact minimization of a specially locked multi-output NAND direct-wire word with constant residual slack, proving polynomial residual-band exact minimization by a finite proof-carrying package, and checking the generated package through a deterministic executable acceptance and replay stack.

The proof route is organized around the implication

$$\begin{aligned} &\text{accepted PCCPack over locked NAND is valid and polynomial} \\ \Longrightarrow\ &\text{polynomial exact residual-band minimization} \\ \Longrightarrow\ &SAT \in P \Longrightarrow P = NP. \end{aligned}$$

The final generated-package certificate is

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept},$$

and the final theorem recorded by the accepted proof report is

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept} \Longrightarrow P = NP.$$

The release custody for this proof report separates source/checker provenance from the durable artefact bundle. The hardened source/checker revision is the annotated git tag

`final-pnp-proof-report-hardened-8b45da4`

at commit

`8b45da4ed604a709d244c35acb886c5eee0889cd`.

The generated artefact JSON commit before metadata sealing is

`c3d3915f7c37737924377076b9216efd2233037f`.

The durable sealed artefact release is the annotated git tag

`final-pnp-proof-report-artifacts-hardened-8b45da4-sealed`.

The materialized proof-report bundle is

`proof-artifacts/final-pnp-proof-report-hardened-8b45da4/`.

1.2 Claim boundary

The claim boundary is the accepted, replayable finite artefact stack culminating in `CheckFinalPNPProofReport0`. The mathematical theorem is asserted under the explicit checker trust model: the finite package, acceptance run, replay, final certificate, release gate, and proof report are the objects being checked, and the public claim is made only because the final proof report accepts.

Theorem 1.1 (Final accepted finite-certificate theorem)**.** *Let* $P_{gen} = \mathrm{GeneratePCCPack}()$. *In the sealed revision, the executable artefact stack records*

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(P_{gen}) = \mathsf{accept}, \qquad \mathrm{CheckAcceptRun}_{\mathrm{exp}}(P_{gen}) = \mathsf{accept}, \qquad \mathrm{ReplayAcceptRun}_{\mathrm{exp}}(P_{gen}) = \mathsf{accept},$$

and the final proof report accepts the public theorem statement

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept} \Longrightarrow P = NP.$$

Therefore SAT is decidable in polynomial time and $P = NP$.

Proof. Sections 2–19 prove the locked NAND residual-slack certificate route: an accepted explicit package yields polynomial exact residual-band minimization and hence SAT in polynomial time. Sections 20 and 23 identify the generated package, deterministic acceptance run, replay, final certificate, release gate, and proof-report artefacts. The final accepted report records accepted `CheckPCCPackexp0`, accepted `CheckAcceptRun0`, accepted replay, accepted final verdict, accepted final certificate, accepted release audit, accepted final release gate, and accepted proof report. The report's theorem field has antecedent $\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept}$, consequent $P = NP$, and conditional flag true. Applying the package-sufficiency theorem to the accepted generated package gives $SAT \in P$. Since SAT is NP-complete, $P = NP$. □

1.3 Central scale correction

The descent scale is residual slack, not root circuit size. For a circuit C, define

$$\Lambda(C) = |C| - \mu(C),$$

where $\mu(C)$ is the minimum equivalent circuit size in the chosen framework. For a compatible support $U \subseteq C$, let $F_{C,U}$ be the induced open carrier-tagged function and define

$$g_C(U) = |U| - \mu^*(F_{C,U}).$$

The global slack law is

$$g_C(U) \leq \Lambda(C).$$

Thus one verified unit gain decreases residual slack by at least one. Locked NAND SAT instances constructed below satisfy

$$\Lambda(W_\varphi^{NAND}) \leq 4.$$

2 Base direct-wire framework

2.1 Direct-wire words

A carrier is a tuple

$$L = (\partial L, \iota L, \operatorname{prof} L),$$

where ∂L is an ordered boundary tuple, ιL is an ordered interface or output tuple, and $\operatorname{prof} L$ records carrier constants, prefix-tail slots, origin slots, finite-kernel slots, obligation slots, direction records, saturation records, budget records, and charge slots.

A NAND direct-wire word in carrier L is a sequence

$$W = (g_1, \ldots, g_m, o),$$

where each gate is

$$g_j = \mathsf{N}(s_j, t_j) = \neg(s_j \wedge t_j),$$

and each source is a boundary coordinate, carrier constant 0 or 1, or an earlier gate output. The output tuple o is ordered.

The semantics map is

$$[[-]]_L : \mathsf{DWWord}(L) \to \mathsf{OpenFun}(L).$$

For an open function F, define

$$\mu_L^*(F) = \min\{|W| : [[W]]_L = F\}.$$

2.2 Compatible supports and replacement

A set $U \subseteq C$ is compatible if every wire entering U from outside is listed as a boundary port, every wire leaving U is listed as an interface port, and all internal gate sources are internal to U or listed boundary sources. Extraction gives

$$(L_U, W_U) = \operatorname{Extract}(C, U),$$

with

$$[[W_U]] = F_{C,U}.$$

Lemma 2.1 (Compatible replacement)**.** *If U is compatible and R is an open word in the same carrier with $[[R]] = F_{C,U}$, then replacing U by R preserves the global function of C.*

Lemma 2.2 (Global slack law)**.** *For every compatible support $U \subseteq C$,*

$$g_C(U) \leq \Lambda(C).$$

Proof. Replace U by a minimum open realization of $F_{C,U}$. The resulting circuit computes the same global function and has size $|C| - g_C(U)$. Hence $\mu(C) \leq |C| - g_C(U)$, which rearranges to the claim. □

3 Saturated support calculus and square closure

3.1 Raw, completed, and saturated supports

Let $\mathcal{P}(C)$ be the primitive record universe of C: NAND gates, boundary and interface records, carrier records, origin records, kernel records, obligation records, prefix-tail records, budget records, direction records, saturation records, and charge records.

A raw support is a finite subset

$$U^{raw} \subseteq \mathcal{P}(C).$$

A completed support is

$$\mathrm{Comp}(U^{raw}) = (U, \partial U, \iota U, \mathcal{F}_U),$$

where completion records incoming wires as boundary coordinates, outgoing wires as ordered interface coordinates, and every touched profile record as internal, materialized, or exported as a frontier obligation.

Define a finite closure operator

$$\mathrm{Sat}_C : \mathcal{P}(C) \to \mathcal{P}(C).$$

It is generated by gate-source closure, interface-consumer closure, origin closure, finite-kernel closure, obligation closure, prefix-tail closure, budget and saturation closure, direction closure, and charge closure.

Theorem 3.1 (Saturation closure)**.** *For all $U, V \subseteq \mathcal{P}(C)$,*

$$U \subseteq \mathrm{Sat}_C(U), \qquad \mathrm{Sat}_C(\mathrm{Sat}_C(U)) = \mathrm{Sat}_C(U),$$

and

$$U \subseteq V \Longrightarrow \mathrm{Sat}_C(U) \subseteq \mathrm{Sat}_C(V).$$

For saturated supports A, B, raw union and intersection are completed by

$$A \vee B = \mathrm{Comp}(\mathrm{Sat}_C(A \cup_{raw} B)),$$

$$A \wedge B = \mathrm{Comp}(\mathrm{Sat}_C(A \cap_{raw} B)).$$

The notation $A \cup B$ and $A \cap B$ is legal in theorem statements only after this saturation and completion step.

Theorem 3.2 (Saturated support square closure)**.** *If A, B are compatible saturated supports, then $A \wedge B$ and $A \vee B$ are compatible saturated supports, unless completion exposes a first obstruction of kind* CritC, Q, E, L*, or a budget, hereditary, selector, X-failure, verified gain, exact route, or strict descent outcome.*

3.2 Support squares and projection compatibility

The accepted support square is

$$\begin{array}{ccc} A \wedge B & \longrightarrow & A \\ \downarrow & & \downarrow \\ B & \longrightarrow & A \vee B. \end{array}$$

The frontier pushout condition is

$$\mathcal{F}_{A \vee B} = \mathcal{F}_A \star_{\mathcal{F}_{A \wedge B}} \mathcal{F}_B,$$

with carrier, origin, kernel, obligation, prefix-tail, direction, saturation, budget, and charge records glued by explicit transport. If projection does not commute with the square, the first failure routes to CritC, Q, E, L, or to semantic, direction, resource, Hall, budget, selector, exact-route, or descent outcomes.

Theorem 3.3 (BN2 square legitimacy)**.** *Every BN2 support square in a no-outcome branch is a legitimate saturated projection-compatible square. Thus $m_i(A), m_i(B), m_i(A \wedge B), m_i(A \vee B), \delta_i(A, B), \Omega(A, B)$, and $d(A)$ are well-defined on the same carrier-compatible square.*

4 Global charge and materializer ownership

4.1 Chargeable objects

For a descendant C, define the charge universe $\mathsf{Ch}(C)$. Allowed chargeable objects are physical NAND gates, exact-prefix materializers, origin materializers, finite-kernel materializers, closed obligation materializers, and charged profile contexts. Ordinary gates have weight 1. A materializer record ρ has weight

$$\mathrm{wt}(\rho) = |Mat(\rho)|.$$

Proof-only records are not in $\mathsf{Ch}(C)$ and cannot feed gates or appear as free Boolean outputs.

The size convention is

$$|C| = \sum_{x \in \mathsf{Ch}(C)} \mathrm{wt}(x).$$

Every chargeable object has a unique owner

$$\mathrm{own}_C : \mathsf{Ch}(C) \to \mathsf{Own}(C).$$

4.2 Touch-to-charge principle and charge laws

For every normalized profile record ρ, a touch predicate $Touch_C(U, \rho)$ is defined. If a support touches a computational profile record, then the record must be a genuine free carrier datum, materialized and charged inside the support, materialized and charged on the exterior frontier, or represented as an open obligation that cannot be used until discharged.

For a traceable normalization step $C_t \to C_{t+1}$,

$$\kappa_t(W) = \sum_{\rho \in Touch_t(W)} \mathrm{wt}(Mat_t(\rho)),$$

and for every support and replacement,

$$|Pull_t(W)| - |W| = |Expand_t(R)| - |R| = \kappa_t(W).$$

For an HN leaf L with blocks B_i,

$$\mathsf{Ch}(U) = \mathsf{Ch}_{fixed}(L) \sqcup \bigsqcup_i \mathsf{Ch}(B_i),$$

so a BWL path p satisfies

$$|R_p| = \kappa_L + \sum_{e \in p} cost(e).$$

Theorem 4.1 (Charge soundness)**.** *If $CheckCharge_{\exp}$ accepts a normalization, HN, budget, splice, BCEL, selector, or neutral-overlay charge certificate, then the relevant size equation is an exact integer identity over uniquely owned computational charges.*

5 Mode projection calculus

5.1 Full and quotient modes

A full carrier is

$$L^1 = (\partial, \iota, prof^1),$$

with

$$prof^1 = (car, orig, ker, obl, pref, dir, sat, bud, chg, front).$$

A quotient carrier is

$$L^0 = \rho L^1,$$

where ρ may forget or coarsen finite profile data. It does not add free Boolean functions.

For a full word W^1, projection ρW^1 keeps the same NAND DAG and applies ρ to profile references. Therefore

$$|\rho W^1| = |W^1|,$$

and

$$[[\rho W^1]]_{L^0} = \rho([[W^1]]_{L^1}).$$

For a saturated support X, define

$$F_X^0 = \rho F_X^1,$$
$$m_1(X) = \mu_1^*(F_X^1), \qquad m_0(X) = \mu_0^*(F_X^0),$$

and projection defect

$$d(X) = m_1(X) - m_0(X).$$

Theorem 5.1 (Projection monotonicity)**.** *For every saturated support X,*

$$m_0(X) \le m_1(X), \qquad d(X) \ge 0.$$

5.2 Mode firewall and transfer identity

A quotient equality is not a full equality. A quotient word can be used constructively in full mode only if it has a full lift and all lost-bit or finite-kernel obligations are discharged through Package E. Otherwise it remains a comparison object used to measure projection defect.

For projection-compatible saturated support squares, define

$$\delta_i(X,Y) = m_i(X) + m_i(Y) - m_i(X \wedge Y) - m_i(X \vee Y),$$

$$\Omega(X,Y) = \delta_0(X,Y) - \delta_1(X,Y).$$

Then

$$\boxed{d(X \vee Y) + d(X \wedge Y) = d(X) + d(Y) + \Omega(X,Y).}$$

6 Verifier-readable proof calculus, normalization, and splice

6.1 Package E: direct-wire verifier

A constructive gain certificate is

$$(C, U, R, \Pi),$$

where $U \subsetneq C$ is compatible, R is an explicit replacement open word, and Π proves

$$[[R]] = F_{C,U}, \qquad |R| \le |U| - 1.$$

The rewrite families are R1 to R9: structural congruence, finite carrier recoding, exact-prefix strip and absorb, latent sharing and fold, finite-kernel bit drop with obligation creation, matched cancellation with full-mode discharge, unary cut realization, origin pullback and promoted latent expansion, and integer incidence accounting.

Only R5 creates obligations. Only R6, R7, and R8 discharge obligations. Final ledgers must have no open obligations. R6 may discharge an obligation only with a full-mode cancellation witness. Projected equality alone is insufficient.

Theorem 6.1 (VerifyDW soundness)**.** *If*

$$\mathrm{VerifyDW}_{\mathrm{exp}}(C, U, R, \Pi) = \mathit{accept},$$

then $U \subsetneq C$ is compatible,

$$[[R]] = F_{C,U}, \qquad |R| \le |U| - 1.$$

Consequently,

$$[[C[U \leftarrow R]]] = [[C]], \qquad \Lambda(C[U \leftarrow R]) \le \Lambda(C) - 1.$$

6.2 Traceable normalization

The normalizer uses traceable operations N1 to N10: topological reorder and renaming, dead bookkeeping deletion, carrier recoding, tuple normalization, exact-prefix metadata absorption, origin normalization, finite-kernel normalization, obligation sorting and cancellation, profile normalization, and gain surfacing.

A step $C_t \to C_{t+1}$ carries

$$Tr_t = (\Phi_t, Pull_t, Expand_t, \kappa_t, \Pi_t).$$

For every compatible support $W \subseteq C_{t+1}$ and replacement R,

$$|Pull_t(W)| - |W| = |Expand_t(R)| - |R|.$$

Thus strict saving transports backward.

Theorem 6.2 (Traceable normalization). *If* $\mathrm{NormalizeOrGain}_{\exp}(C) = Normal(C^\nu, T^\nu)$, *then*

$$[[C^\nu]] = [[C]], \quad |C^\nu| \le |C|,$$

$$\mu(C^\nu) = \mu(C), \quad \Lambda(C^\nu) \le \Lambda(C).$$

Every positive residual witness in C^ν *pulls back to a positive residual witness in* C. *If* $\mathrm{NormalizeOrGain}_{\exp}(C) = Gain(U, R, \Pi)$, *then* $\mathrm{VerifyDW}_{\exp}(C, U, R, \Pi) = accept$.

6.3 Bounded and composed splices

A bounded splice instance consists of a compatible bounded support U, a same-frontier replacement R, symbolic windows ω_U, ω_R, finite transport, materializer ledger, obligation-discharge ledger, and size ledger. It requires ordered frontier equality, bounded truth-vector equality, finite relation equality, profile transport, full-mode obligation closure, and

$$|R| \le |U| - 1.$$

Theorem 6.3 (Bounded splice). *Every accepted bounded splice instance compiles to a ledger* Π *with*

$$\mathrm{VerifyDW}_{\exp}(C, U, R, \Pi) = accept.$$

A composed splice decomposes a long support as

$$U = U_1 \cup \cdots \cup U_m$$

with declared seams. Seam compatibility requires ordered interface-boundary agreement, carrier agreement, origin and kernel agreement, full-mode obligation compatibility, direction and saturation agreement, and unique budget and charge ownership.

Theorem 6.4 (Composed splice). *If a composed splice instance is seam-compatible, charge-valid, and at least one local or seam component has strict saving, then the generated global ledger satisfies* $\mathrm{VerifyDW}_{\exp}$.

7 Finite local routers: FT, X, BC, and UN

7.1 Finite-table engine FT

A finite local table is built over symbolic windows

$$\omega = (D, \partial, \iota, o, H, prof, guard, relSpec, payloadSlots),$$

where D is a bounded NAND DAG, H is a finite handle-variable tuple, and $prof$ lies in finite profile alphabets. A schema is

$$S = (k, \beta, o_{max}, h, \mathcal{P}, \mathcal{A}, \mathcal{G}, \mathcal{C}, \mathsf{NF}, \mathsf{PayloadSpec}).$$

The raw universe is finite. Runtime integers such as D, p, q, M_0, B_1 are arithmetic fields, not finite profile states.

The bounded evaluator enumerates all 2^β formal boundary assignments. Symbolic normalization returns both a code and transport:

$$\mathsf{NF}(\omega) = (\widehat{\omega}, \tau_\omega).$$

The transport preserves truth vectors, relation tables, profile fields, handle slots, output order, source incidence, consumer incidence, dependency edges, charge ownership, obligations, direction, saturation, budget, and payload fields.

Theorem 7.1 (Finite-table coverage). *If* $CheckTable_{\exp}(T, S) = accept$, *then every physical configuration governed by* S *is represented, normalized, classified, and soundly routed by* T.

7.2 Critical-window tables X and FT-X

The finite critical kinds are:

$$\begin{aligned} \text{CritC} &= \text{full row seeking quotient shadow},\\ Q &= \text{quotient row seeking full restoration},\\ E &= \text{exposure-obligation retract},\\ L &= \text{overlap-loss preservation}. \end{aligned}$$

Each row has candidate towers

$$ResTab_\kappa(r) \subseteq DirTab_\kappa(r) \subseteq SemTab_\kappa(r).$$

Empty semantic, direction, or resource candidate sets produce $X2, X3, X4$, respectively, with exhaustive certificates. If resource targets exist, matching is performed fibrewise by projected signature. A Hall witness gives $X1$. A bounded strict replacement emits $A(s)$ and compiles to $\text{VerifyDW}_{\text{exp}}$.

The deterministic route priority for X rows is

$$A(s) \succ X2 \succ X3 \succ X4/\text{CritKind} \succ X1 \succ \text{CritKind} \succ Neutral \succ Downstream.$$

This prevents a row with a bounded gain from being downgraded to an obstruction, and prevents semantic impossibility from masquerading as a weaker kind-specific row.

7.3 Branch-cycle router BC

Package BC handles connected full-span non-unary incidence graphs. It uses a finite transition category with states in

$$\Sigma_{BC} = \Sigma_{car} \times \Sigma_{pref} \times \Sigma_{orig} \times \Sigma_{ker} \times \Sigma_{obl} \times \Sigma_{dir} \times \Sigma_{sat} \times \Sigma_{bud} \times \Sigma_{chg}.$$

Transition labels include full and projected transition relations, projection record, defect, saturation, budget, obligation, charge, size, normalized code, and local certificate.

Composition is partial. Undefined composition emits a first failed coordinate, such as $X2$, $X3$, $X4$, CritC, Q, E, L, budget, HNShape, graph descent, or UnaryWindow. Cycle holonomy and branch tripod CSPs are finite.

Theorem 7.2 (Package BC)**.** *Every governed connected full-span non-unary primitive critical window routes to a verified gain, certified X-failure, HNShape, BudgetShape, graph-core descent, or a governed UnaryWindow.*

7.4 Unary decoder UN

A unary path is

$$P = (v_0, e_1, v_1, \dots, e_m, v_m).$$

The finite state alphabet is

$$\Sigma_{UN} = \Sigma_{car} \times \Sigma_{pref} \times \Sigma_{orig} \times \Sigma_{ker} \times \Sigma_{obl} \times \Sigma_{dir} \times \Sigma_{sat} \times \Sigma_{bud} \times \Sigma_{act} \times \Sigma_{chg}.$$

Each edge has full and projected transition relations T_i^1, T_i^0. Forward and backward full sets are

$$\begin{aligned} F_0 = B_0, qquad F_i = T_i^1(F_{i-1}),\\ R_m = B_m, \qquad R_{i-1} = (T_i^1)^{-1}(R_i). \end{aligned}$$

A full lift exists iff

$$F_m \cap B_m \neq \varnothing.$$

If a projected lift exists but no full lift exists, UN outputs a canonical minimal blocked interval $[a, b]$ with

$$T^1_{[a,b]}(F_a) \cap R_b = \varnothing,$$

and

$$T^0_{[a,b]}(\rho F_a) \cap \rho R_b \neq \varnothing.$$

Theorem 7.3 (Package UN)**.** *Every governed connected full-span unary window routes to a* $\text{VerifyDW}_{\text{exp}}$*-accepted gain, certified X-failure, HNShape, BudgetShape, SelectorSeed, BCLeak, strict unary descent, or clean full-span HSC-spine token.*

BC and UN never import each other. BC emits 'UnaryWindow' and UN emits 'BCLeak' as *EmitToken* route objects only. Route scheduling consumes such tokens later.

8 Hereditary normal forms, HResolve, and BUD

8.1 Hereditary package HN

Hereditary leaves have type

$$\tau \in \{pair, tripod, spine, nf\},$$

corresponding to HSC-pair, HSC-tripod, HSC-spine, and hereditary non-flatness. A certificate is

$$L = (\tau, U, \Gamma, shape, blocks, frontier, charge, \Pi_{struct}).$$

The block decomposition is

$$U = B_1 \cup \cdots \cup B_m.$$

The BWL layered graph minimizes

$$(cost, resrank, frdev, dwcode).$$

Theorem 8.1 (BWL core exactness)**.** *The BWL shortest path computes the exact minimum inside the certified hereditary grammar and reconstructs a frontier-faithful realization.*

Each hereditary type has local normal rules LN1 to LN10. Critical pairs are generated over bounded overlaps. Each bounded peak is joined or emits VerifyDWGain, BudgetShape, HNShape, SelectorSeed, LeafDescent, or a finite critical row. Termination plus critical-pair coverage gives confluence on no-exit components.

ParseOrExit completeness states that every same-frontier equivalent open word either parses into the hereditary grammar after no-exit normalization, or activates a high-priority exit.

Theorem 8.2 (Leaf-tightness)**.** *If L is accepted and no HN high-priority exit is active, the BWL output R_L satisfies*

$$[[R_L]] = F_{C,U}, \qquad \mathcal{F}(R_L) = \mathcal{F}(U), \qquad |R_L| = \mu^*(F_{C,U}).$$

8.2 Global hereditary resolver

HResolve enumerates accepted proper hereditary leaves and full-span side-closed HSC-spines. Full-span spines are solved first. Proper leaves are assembled into a maximal H-disjoint family. H-disjointness includes support, frontier, origin, kernel, obligation, prefix-tail, charge, and interface noninterference.

The NoHereditary sidecar is strong and blocker-indexed. Each hereditary candidate has an entry H0 through H4: invalid, solved, budget-blocked, selector-blocked, or neutral/dead.

Theorem 8.3 (HResolve)**.** *The global hereditary resolver has three sound outputs:*

$$HResolve_{\exp}(C) = Gain(U, R, \Pi) \Rightarrow \mathrm{VerifyDW}_{\exp}(C, U, R, \Pi) = accept,$$

$$HResolve_{\exp}(C) = Minimum(C, M, \mathcal{M}) \Rightarrow [[M]] = [[C]] \text{ and } |M| = \mu(C),$$

$$HResolve_{\exp}(C) = NoHereditary(C, \mathcal{N}_H),$$

where $\mathcal{N}_H$ is a total sidecar over governed hereditary candidates.

8.3 Budget resolver

Each budget row has a budget vector

$$bud(r) = (M_0(r), B_1(r)) \in \mathbb{Z}^2_{\geq 0}.$$

For a saturated hull H,

$$M_0(H) = \sum_{r \in H} M_0(r), \qquad B_1(H) = \sum_{r \in H} B_1(r).$$

The bounds $M_0^{max}(H), B_1^{max}(H)$ are certified upper envelopes over a finite neutral no-outcome grammar, computed by dynamic programming.

BudgetResolve routes active certificates through proper, disconnected full-span, branch-cycle, unary, full-spine, HN, packet, exact-route, or descent routes. It emits a strong NoBudget sidecar. Each budget candidate has entry B0 through B4: invalid, within envelope, hereditary-blocked, selector-blocked, or solved/routed.

Theorem 8.4 (BudgetResolve). *BudgetResolve has three sound outputs:*

$$BudgetResolve_{\exp}(C) = Gain(U, R, \Pi) \Rightarrow \mathrm{VerifyDW}_{\exp}(C, U, R, \Pi) = accept,$$
$$BudgetResolve_{\exp}(C) = Minimum(C, M, \mathcal{M}) \Rightarrow [[M]] = [[C]], \quad |M| = \mu(C),$$
$$BudgetResolve_{\exp}(C) = NoBudget(C, \mathcal{N}_B),$$

where $\mathcal{N}_B$ is a total sidecar over governed budget candidates.

HN and BUD may reference each other only through sidecar blocker edges. Actual simultaneous acyclicity is checked later by HB.

9 Frontier-faithful comparison and strong neutral overlay

9.1 Strong neutral overlay

A saturated hull has full frontier

$$\mathcal{F}_H = (\partial_H, \iota_H, car_H, orig_H, ker_H, obl_H, pref_H, dir_H, sat_H, bud_H, chg_H).$$

A replacement is frontier-faithful if it preserves this ordered frontier under explicit transport.

A strong neutral overlay between same-frontier full-mode open direct-wire words H and R is a bijection

$$M : \mathsf{Ch}(H) \to \mathsf{Ch}(R)$$

that preserves weights, type, truth vectors, relation tables, carrier, origin, kernel, obligation, prefix, direction, saturation, budget, charge records, ordered frontier, dependency edges, source-port incidence, and consumer-port incidence.

Theorem 9.1 (Strong neutral overlay rigidity). *If (H, R) admits a strong neutral overlay, then*

$$|H| = |R|.$$

Moreover, H and R are direct-wire congruent up to finite carrier/profile transport.

9.2 FF-LOSS and frontier-faithful comparison

Let

$$R^{(0)} \to R^{(1)} \to \cdots \to R^{(T)}$$

be the frontier-closure sequence for a cheaper comparison word. A first-loss atom is the first added frontier materializer whose charge destroys strict saving relative to H. Its minimal loss cone is

$$\mathcal{K}(a) = \mathrm{Comp}(\mathrm{Sat}(Dep^{-}(a) \cup Witness(a))).$$

Theorem 9.2 (FF-LOSS). *Every first saving-destroying frontier closure atom routes to one of*

$$\mathrm{CritC}, Q, E, L, X1, X2, X3, X4,$$

or to a verified gain, BC, UN, HN, BUD, selector, exact route, or strict descent. If the loss cone is neutral-exact, it routes to strict residual-rank descent or exact route.

Theorem 9.3 (Frontier-faithful comparison reduction). *In a no-outcome branch, every full-positive witness has a cheaper same-frontier, frontier-faithful full-mode comparison word. If frontier closure fails or destroys saving, FF-LOSS routes the first failed or first-loss coordinate.*

10 Residual witness normal form

10.1 Terminal whole-carrier bridge

For the whole circuit C, define the terminal full-carrier function

$$F_C := F_C^{top,1}.$$

This terminal carrier preserves global Boolean semantics and full-mode admissibility, but it does not force all internal origin, kernel, prefix, obligation, budget, charge, and proof records of C to be externally preserved semantic data.

10.1.1 Terminal MuBridge proof obligations

The accepted residual-band theorem records the terminal bridge as five checker-visible obligations:

```
terminalCarrierPreservesSemantics
terminalizationSizePreserving
closedFullWordRealizesCircuit
quotientEqualityNotConstructive
wholeSpanCheaperImpliesStrictDescent
```

The first three fields state that the terminal carrier represents exactly the whole-circuit Boolean function in full mode and that the two translations between whole circuits and closed full words preserve size. The fourth field is the mode firewall at this boundary: a quotient equality may be used as a comparison object but is never a constructive full-mode replacement. The fifth field connects a cheaper whole-span closed full word to strict residual descent, rather than to an unverified local gain.

Theorem 10.1 (RW-MuBridge, terminal form)**.** *For every normalized whole circuit C,*

$$\mu(C) = \mu_1^*(F_C^{top,1}).$$

Proof. The inequality $\mu_1^*(F_C^{top,1}) \le \mu(C)$ is witnessed by terminalizing any whole-circuit realization of C. Terminalization keeps the Boolean output function, orders the boundary and output tuple as the terminal full carrier, records only admissible full-mode profile data, and does not expose proof, origin, kernel, budget, charge, or obligation records as free semantic outputs. The field `terminalizationSizePreserving` records that this map introduces no NAND gate and drops no computational gate, so the size is preserved.

Conversely, let R be a closed full word realizing $F_C^{top,1}$. The field `closedFullWordRealizesCircuit` certifies that R can be interpreted as a whole circuit $\mathrm{Real}_C(R)$ with the same Boolean input/output semantics and the same number of NAND gates. Full-mode carrier records are either boundary constants, charged materializers, or discharged obligations; none are free proof data. Hence $\mu(C) \le \mu_1^*(F_C^{top,1})$.

The two inequalities give equality. The quotient firewall is used only to prevent an invalid converse: a quotient-mode word realizing $\rho F_C^{top,1}$ is not a full-mode realization of $F_C^{top,1}$ unless it carries a checked full lift and all lost-bit or finite-kernel obligations are discharged. Therefore quotient equality cannot be used constructively in the terminal bridge. □

10.2 Full-span policy

A full-span witness $H = C$ is legal as a residual witness, but it cannot be emitted as a local $\mathrm{VerifyDW}_{\mathrm{exp}}$ gain. A cheaper whole-span realization gives strict residual descent; an exact whole-span realization gives exact route.

Lemma 10.2 (Whole-span cheaper word gives descent)**.** *If R is a closed full word with $[[R]] = F_C$ and $|R| < |C|$, then $C' = \mathrm{Real}_C(R)$ satisfies $[[C']] = [[C]]$ and*

$$\Lambda(C') < \Lambda(C).$$

10.3 SaturatePositive and RankWF

A residual witness candidate H^0 may not be saturated. Run deterministic saturation and completion:

$$H^0 \to H^1 \to \cdots \to H^T = \mathrm{Comp}(\mathrm{Sat}_C(H^0)).$$

Let

$$\Phi_t = |H^t| - \mu_1^*(F_{H^t}^1), \qquad D_t = \mu_1^*(F_{H^t}^1) - \mu_0^*(F_{H^t}^0).$$

Transparent saturation steps add the same forced full-mode cost to the support and to the minimum, and add no more quotient cost than full cost. Thus full positivity is preserved and projection positivity does not decrease.

10.3.1 SaturatePositive proof obligations

The accepted residual-band theorem records the positivity-preservation audit as five checker-visible obligations:

```
transparentSaturationCostBalanced
interfaceExposureRoutesToE
originKernelObligationClosureRouted
projectionPositivityNotLostSilently
firstNontransparentStepRecorded
```

These fields say that every transparent saturation step adds the same forced full-mode cost to the support and to the full minimum, that exposure-only growth is routed to E unless it is a zero-cost retract, that origin, kernel, and obligation closure failures route instead of silently changing the witness, that projection positivity cannot disappear without a named route, and that the first nontransparent saturation step is recorded as the deterministic failure coordinate.

Theorem 10.3 (RW-SaturatePositive)**.** *Saturating a positive residual witness either preserves full or projection positivity, or routes the first positivity-loss coordinate to* $\mathrm{CritC}, Q, E, L, X$*, gain, BC, UN, HN, BUD, selector, exact route, or strict descent.*

Proof. Run the deterministic saturation chain $H^0 \to \cdots \to H^T$. At each step compare

$$\Phi_t = |H^t| - \mu_1^*(F^1_{H^t}), \qquad D_t = \mu_1^*(F^1_{H^t}) - \mu_0^*(F^0_{H^t}).$$

For a transparent closure step, the added record is a forced full-mode materializer or profile completion whose charge is uniquely owned. The same charge is added to the support size and to every full-mode realization of the completed function. Thus Φ_t is preserved. Projection cost cannot increase more than the full cost, so D_t does not decrease. This is the obligation `transparentSaturationCostBalanced`.

If a closure step is not transparent, the checker records the first failed coordinate. Interface exposure is the delicate case: adding a required outgoing coordinate can impose a new semantic obligation without adding a physical support gate. Such a step routes to E unless the exposed value is a certified zero-cost retract already present on the frontier. Origin, kernel, and obligation closures are treated the same way: an undischargeable or mode-unsafe closure routes to $\mathrm{CritC}, Q, E, L, X$, a package route, a selector, an exact route, a verified gain, or strict descent. These are the obligations `interfaceExposureRoutesToE`, `originKernelObligationClosureRouted`, and `firstNontransparentStepRecorded`.

Therefore, along any no-outcome branch, every saturation step is transparent for the active witness and preserves full positivity or projection positivity. If either positivity measure is lost, the first step at which it is lost is nontransparent and is emitted as a named route. This proves the theorem. The finite residual rank tuple then orders any routed descent by witness type, span type, mode, frontier defect, projection defect, saturation defect, anchor count, charge size, profile size, and canonical code, so repeated routing is well-founded. □

Residual ranks are finite lexicographic tuples over witness type, span type, mode, frontier defect, projection defect, saturation defect, anchor count, charge size, profile size, and canonical code. Therefore the rank order is well-founded.

10.4 BCEL-ready residual witnesses

A BCEL-ready record is

$$\mathrm{BCELReady}(C, H, A, \mathcal{X}, D).$$

It certifies:

1. H is governed, compatible, saturated, completed, and projection-positive:
$$d(H) = D > 0.$$
2. A finite anchor set $A = A(H)$ partitions primitive records.
3. For every $S \subseteq A$,
$$X_S = \mathrm{Comp}(\mathrm{Sat}_C(\bigcup_{a \in S} P_a)).$$
4. The anchor family is Boolean:
$$X_S \wedge X_T = X_{S \cap T}, \quad X_S \vee X_T = X_{S \cup T}.$$
5. All anchor squares are projection-compatible.

6. A is a minimal positive nucleus:

$$d(X_A) = D > 0, \qquad B \subsetneq A \Rightarrow d(X_B) = 0.$$

7. $|A| \geq 2$.
8. For every nonempty proper cut,

$$\Omega(S, A \setminus S) = D.$$

10.4.1 BCEL-ready positive nucleus proof obligations

The accepted residual-band theorem records the BCEL-ready positive nucleus step as checker-visible obligations:

```
positiveResidualWitnessYieldsBCELReady
positiveResidualWitnessExists
finiteAnchorSetExtracted
booleanAnchorAlgebraOrRoute
minimalPositiveNucleus
properCutConstantEquation
anchorSizeAtLeastTwo
bcelAnchorAlgebraBooleanOrRoutes
```

The first field is the public summary obligation: positive residual slack must yield a BCEL-ready hull unless an earlier route fires. The next six fields decompose the construction. A positive residual witness exists after terminalization and saturation, a finite anchor set is extracted from the touched primitive records, non-Boolean anchor algebra routes instead of remaining silent, a minimal positive nucleus is chosen by finite descent, every proper cut has the constant-cut equation, and the nucleus has at least two anchors. The final field records the firewall: Boolean-anchor failures, saturation failures, and proper-cut failures are routes, not silent assumptions.

Theorem 10.4 (RW-BCELReady)**.** *If C is normalized, $\Lambda(C) > 0$, and no verified gain, exact route, named outcome, selector route, or strict residual descent is active, then there exists a BCEL-ready projection-positive hull.*

Proof. By Terminal MuBridge, $\Lambda(C) > 0$ gives a positive residual witness in the terminal full carrier. If the witness is whole-span and cheaper, the whole-span policy emits strict residual descent, contrary to the no-outcome hypothesis. Otherwise the witness is a proper or governed residual witness. By SaturatePositive, deterministic saturation and completion either preserve full or projection positivity, or route the first positivity-loss coordinate to a named outcome, verified gain, exact route, selector, or strict descent. Since those routes are absent, there is a saturated completed projection-positive hull H with

$$d(H) = D > 0.$$

This is the obligation `positiveResidualWitnessExists`.

The primitive record universe touched by H is finite. Partition it into canonical anchor blocks by frontier incidence, transport profile, origin/kernel/obligation footprint, activation code, and saturation-closure dependency. The resulting finite anchor set $A = A(H)$ defines

$$X_S = \mathrm{Comp}(\mathrm{Sat}_C(\bigcup_{a \in S} P_a))$$

for every $S \subseteq A$. This is `finiteAnchorSetExtracted`. If some intersection, union, transport, projection, or saturation square fails to satisfy the Boolean anchor law

$$X_S \wedge X_T = X_{S \cap T}, \qquad X_S \vee X_T = X_{S \cup T},$$

the first failed coordinate is a named route by the same saturation and mode-firewall rules. Under the no-outcome hypothesis no such failure remains, so the anchor algebra is Boolean; this is `booleanAnchorAlgebraOrRoute` and `bcelAnchorAlgebraBooleanOrRoutes`.

Among all nonempty anchor subsets with positive projection defect choose one minimal by inclusion and then by the residual rank tuple. This finite choice is valid because A is finite. Let the chosen subset again be called A. Then

$$d(X_A) = D > 0, \qquad B \subsetneq A \Rightarrow d(X_B) = 0,$$

which is `minimalPositiveNucleus`. A singleton positive nucleus would be a local positive atom with no proper cut; the finite local routers, exposure router, hereditary/budget routers, selector seed, exact route, or strict descent route would fire. Since no such route is active, $|A| \geq 2$; this is `anchorSizeAtLeastTwo`.

For every nonempty proper $S \subsetneq A$, apply the transfer identity to the Boolean anchor square $(X_S, X_{A\setminus S})$. Minimality makes both proper sides have zero projection defect, while the whole nucleus has defect D. Therefore the projection excess across the cut is constant:

$$\Omega(S, A \setminus S) = D.$$

If the equality failed, the first failing square or transport coordinate would be routed as a named outcome, selector, exact route, gain, or strict descent. Hence the constant-cut equation holds in the no-outcome branch; this is `properCutConstantEquation`. The tuple $(C, H, A, \mathcal{X}, D)$ is therefore BCEL-ready and projection-positive. □

11 BN2 through BN6 and Package C

11.1 BN2: side-tight coherent optima

For a legal saturated projection-compatible square

$$\begin{array}{ccc} I & \longrightarrow & A \\ \downarrow & & \downarrow \\ B & \longrightarrow & U, \end{array}$$

define

$$\delta_i(A, B) = m_i(A) + m_i(B) - m_i(I) - m_i(U).$$

A square-coherent tuple is

$$\mathcal{X} = (X_I, X_A, X_B, X_U, \lambda_A, \lambda_B, \lambda_U, \Theta).$$

Its slacks are

$$\epsilon_S = |X_S| - m_i(S).$$

Its incidence value is

$$v_{\mathcal{X}} = |X_A| + |X_B| - |X_I| - |X_U|.$$

Therefore

$$v_{\mathcal{X}} = \delta_i(A, B) + \epsilon_A + \epsilon_B - \epsilon_I - \epsilon_U.$$

A coherent tuple is side-tight iff every $\epsilon_S = 0$.

11.1.1 BN2 side-tight coherent optima proof obligations

The accepted residual-band theorem records the BN2 coherent-optimum step as four checker-visible obligations:

```
sideTightOnlyNoOverclaim
fourCornerOptimaCarrierCompatible
sideTightCompletionExists
tightBasisValueEqualsDelta
```

The first field is the no-overclaim discipline: BN2 may read $\delta_i(A, B)$ only from side-tight bases, not from arbitrary coherent tuples. The second field states that the four corner optima are compared over the same transported square carrier. The third field asserts that every legal saturated projection-compatible square in a no-outcome branch has a side-tight completion. The fourth field records the value identity $v_G = \delta_i(A, B)$ for a side-tight basis.

Theorem 11.1 (BN2-CoherentOptimum)**.** *In a no-outcome branch, every legal saturated projection-compatible square has a side-tight square-coherent tuple in each mode i. For every side-tight basis,*

$$v_G = \delta_i(A, B).$$

Consequently,

$$\delta_i(A, B) = \max_{G \in \mathcal{G}_i^{tight}(A,B)} v_G.$$

Proof. Fix a legal saturated projection-compatible square in mode i. Legality means that the four supports I, A, B, U have already passed saturation, completion, frontier transport, origin/kernel/obligation transport, charge transport, and mode-firewall checks. Therefore the four open functions F_I, F_A, F_B, F_U live in carrier-compatible corners of the same square, and the optima $m_i(I), m_i(A), m_i(B), m_i(U)$ can be compared without mixing frontiers. This is `fourCornerOptimaCarrierCompatible`.

Choose minimum realizers X_I, X_A, X_B, X_U for the four corners, and transport them along the square interfaces. If some transport, frontier order, profile field, charge owner, or obligation discharge cannot be made coherent, the first failing coordinate is a named route, verified gain, exact route, selector, or strict descent; in a no-outcome branch no such failure remains. The side-tight completion field records that the surviving coherent tuple can be chosen with

$$|X_S| = m_i(S) \qquad (S \in \{I, A, B, U\}).$$

Thus all slacks $\epsilon_S = |X_S| - m_i(S)$ vanish. This is `sideTightCompletionExists`.

For any square-coherent tuple the bookkeeping identity is

$$v_{\mathcal{X}} = \delta_i(A, B) + \epsilon_A + \epsilon_B - \epsilon_I - \epsilon_U.$$

Substituting $\epsilon_I = \epsilon_A = \epsilon_B = \epsilon_U = 0$ for a side-tight tuple gives

$$v_G = \delta_i(A, B).$$

This is `tightBasisValueEqualsDelta`. Conversely, if a coherent tuple is not side-tight, the extra slack terms need not vanish, and its incidence value is not licensed as $\delta_i(A, B)$. Therefore BN2 never infers the merge defect from arbitrary coherent bases; it uses only side-tight bases. This is `sideTightOnlyNoOverclaim`. Since a side-tight basis exists in every no-outcome legal square, maximizing over the finite side-tight basis family returns $\delta_i(A, B)$. □

11.2 BN3: simultaneous finite request envelopes

For each mode i, BN3 constructs a finite weighted request system

$$(P_i, \leq_i, \nu_i, D_i).$$

A unit $p \in P_i$ has a stable request predicate

$$r_p^{(i)}(T) = 1_{p \in D_i(T)}.$$

It is represented by a minimal-consumer antichain

$$\mathcal{M}_p^{(i)} = \min_{\subseteq}\{T \subseteq A : T \neq \varnothing, r_p^{(i)}(T) = 1\}.$$

Then

$$r_p^{(i)}(T) = 1 \iff \exists M \in \mathcal{M}_p^{(i)}\ [M \subseteq T].$$

11.2.1 BN3 finite request-envelope proof obligations

The accepted residual-band theorem records the BN3 request-envelope step as five checker-visible obligations:

```
requestPredicatesStable
minimalConsumerAntichainsExact
jointSideTightRealizability
runtimeIntegersSeparatedFromFiniteState
incidenceAtomsAccountedExactlyOnce
```

The first field states that each request predicate is stable under saturation, transport, frontier order, profile recoding, and mode projection. The second says that the minimal-consumer antichain represents the predicate exactly. The third is the simultaneous-realizability requirement: the envelope is not merely a collection of individually possible requests, but is realized in one side-tight BN2 basis. The fourth prevents schedule-sized runtime integers from being smuggled into finite profile state. The fifth records that every cut-incidence atom is counted once, with no missing atom and no duplicate charge.

Theorem 11.2 (BN3-SimultaneousEnvelope)**.** *For each mode i, there exists a finite weighted request system such that for every proper cut S,*

$$\delta_i^{cut}(S, A \setminus S) = \sum_{p \in P_i} \nu_i(p) r_p^{(i)}(S) r_p^{(i)}(A \setminus S).$$

The request predicates are stable, monotone, vanish on the empty set, and jointly realizable in one side-tight BN2 basis.

Proof. Fix a mode i and a BCEL-ready positive nucleus with anchor set A. By BN2, every legal saturated projection-compatible square has carrier-compatible side-tight optima, and the value of a side-tight basis equals the corresponding merge defect. For every primitive incidence atom that can contribute to a cut value, form a request unit p with weight $\nu_i(p)$. Its predicate $r_p^{(i)}(T)$ says that the side indexed by $T \subseteq A$ consumes the incidence atom in the transported side-tight basis.

The request predicate depends only on finite row-key data: transported frontier signature, projected semantic signature, activation code, origin/kernel/obligation footprint, charge owner, budget cell, profile class, and rank tag. Saturation and transport preserve these fields, and any unstable transport, profile recoding, frontier mismatch, mode-firewall failure, or charge-owner mismatch routes to a named outcome, verified gain, exact route, selector, or strict descent. Hence in a no-outcome branch the predicate is stable and monotone, and it vanishes on $\varnothing$. This is `requestPredicatesStable`.

For each request unit, define

$$\mathcal{M}_p^{(i)} = \min_{\subseteq} \{T \subseteq A : T \neq \varnothing,\ r_p^{(i)}(T) = 1\}.$$

Since A is finite, this antichain is finite. Monotonicity gives the upward closure identity

$$r_p^{(i)}(T) = 1 \iff \exists M \in \mathcal{M}_p^{(i)}\ [M \subseteq T].$$

If a consumer set is omitted, nonminimal, or duplicated, the finite antichain checker exposes the first mismatch. Thus the antichain is exact; this is `minimalConsumerAntichainsExact`.

The envelope is then assembled simultaneously from one side-tight BN2 basis. The request units are not certified one at a time with unrelated witnesses; instead, a joint side-tight tuple supplies all corner optima, all transported incidence atoms, and all request activations in one compatible square. Therefore requests that are individually possible but mutually inconsistent cannot be used. This is `jointSideTightRealizability`.

The runtime integers that occur in budget values, charge multiplicities, envelope weights, table bounds, and Presburger certificates remain arithmetic fields. They are not added to the finite profile alphabet and are compared only through the accepted arithmetic-cell certificates. This keeps the request universe finite under the fixed schedule and is `runtimeIntegersSeparatedFromFiniteState`.

Finally, each cut-incidence contribution has a unique atom key, owner, side-tight basis coordinate, and request unit. The incidence ledger partitions the atom set: every atom contributing to $\delta_i^{cut}(S, A \setminus S)$ appears exactly once in the sum, and no atom whose predicate is inactive on one side of the cut contributes. This gives

$$\delta_i^{cut}(S, A \setminus S) = \sum_{p \in P_i} \nu_i(p) r_p^{(i)}(S) r_p^{(i)}(A \setminus S)$$

for every proper cut. This is `incidenceAtomsAccountedExactlyOnce`. Raw individual requestability is therefore insufficient; BN3 uses stable requestability plus joint realizability. □

11.3 BN4: activation-exact cancellation

Define the active consumer antichain

$$\mathcal{A}_p^{(i)} = \{M \in \mathcal{M}_p^{(i)} : \exists N \in \mathcal{M}_p^{(i)},\ M \cap N = \varnothing\}.$$

If $\mathcal{A}_p = \varnothing$, then p is cut-silent. The activation is

$$\alpha_p(S) = r_p(S) r_p(A \setminus S).$$

Activation equality is certified by active-antichain equality:

$$\alpha_p = \alpha_q \iff \mathcal{A}_p = \mathcal{A}_q.$$

No package may certify activation equality by enumerating all cuts.

11.3.1 BN4 activation-exact cancellation proof obligations

The accepted residual-band theorem records the activation-cancellation step as five checker-visible obligations:

```
activationByActiveAntichain
activationEqualityWithoutCutEnumeration
sameKeyCancellationExact
noOppositeSignSameKeyResidual
integerMassLedgerExact
```

The first field says that the activation function of a request is represented by its active minimal-consumer antichain. The second forbids cut enumeration as the equality proof. The third requires cancellation only between positive and negative cells with the same projected semantic signature, active-antichain code, and projected transport type. The fourth records that no positive and negative residual cells with the same activation-exact key survive. The fifth records that all multiplicities and cancellations are checked in the integer mass ledger.

A BN4 key is

$$\kappa(p) = (\bar{\sigma}(p), \mathrm{ActCode}(p), \tau(p)),$$

where $\bar{\sigma}$ is the projected semantic signature and τ is projected transport type.

Theorem 11.3 (BN4-ActivationExact)**.** *For every proper cut,*

$$\Omega(S, A \setminus S) = \sum_{c_\kappa > 0} c_\kappa \alpha_\kappa(S) - \sum_{c_\kappa < 0} (-c_\kappa)\alpha_\kappa(S).$$

After cancellation, no positive and negative residual cells share the same activation-exact key.

Proof. For a request unit p, the cut activity is

$$\alpha_p(S) = r_p(S) r_p(A \setminus S).$$

Since each request predicate is the upward closure of its exact minimal-consumer antichain, α_p is determined by the disjoint active pairs of minimal consumers. The active antichain

$$\mathcal{A}_p^{(i)} = \{M \in \mathcal{M}_p^{(i)} : \exists N \in \mathcal{M}_p^{(i)},\ M \cap N = \varnothing\}$$

therefore determines the same activation function on every proper cut. If two requests have different active antichains, choose a minimal set appearing in one and not the other and extend it by a disjoint active partner; the corresponding cut separates the two activations. Hence activation equality is exactly active-antichain equality. This is `activationByActiveAntichain` and `activationEqualityWithoutCutEnumeration`. The certificate compares canonical active-antichain codes, not the exponentially many cuts.

Each request atom carries a projected semantic signature, an activation code, a projected transport type, an integer mass, and a ledger owner. Cells are grouped by the BN4 key

$$\kappa(p) = (\bar{\sigma}(p), \mathrm{ActCode}(p), \tau(p)).$$

Only cells with the same key compute the same cut function and may cancel. Cells with different semantic signatures, transport types, or active-antichain codes are kept separate even if they agree on a sampled cut set. This is `sameKeyCancellationExact`.

Within a fixed key, the integer mass ledger sums all positive and negative cells using the signed weights supplied by BN3. The arithmetic-cell certificate checks that the signed total is exact, that no owner is counted twice, and that cancellation removes precisely the common positive and negative mass. After cancellation, either a nonnegative residual key remains, a negative residual key is sent to BN5/PkgC localization, or the key is cut-silent. The cancellation pass rejects any state with surviving positive and negative residual cells of the same key. This is `noOppositeSignSameKeyResidual` and `integerMassLedgerExact`.

Substituting the grouped signed masses into the BN3 incidence expansion gives, for every proper cut,

$$\Omega(S, A \setminus S) = \sum_{c_\kappa > 0} c_\kappa \alpha_\kappa(S) - \sum_{c_\kappa < 0} (-c_\kappa)\alpha_\kappa(S).$$

The formula is valid for all proper cuts because the key equality is activation-exact, not sampled. This proves BN4. □

11.4 BN5: full-shadow localization

For a negative full residual key κ, refine masses into unit atoms and form a shadow graph

$$\mathcal{B}_\kappa = (L_\kappa, R_\kappa, E_\kappa),$$

where left vertices are full atoms and right vertices are quotient candidate shadows with the same activation-exact key. Hall deficits and first unmatched full atoms localize to CritC rows or X-failures.

11.4.1 BN5 full-shadow localization proof obligations

The accepted residual-band theorem records BN5 localization as checker-visible obligations:

```
negativeFullResidualLocalized
hallDeficitRoutesNamedOutcome
unmatchedShadowNotSilent
```

The first field says that every cut-active negative full residual key is represented by a full-shadow matching problem. The second says that every Hall deficit is emitted as a named route, not hidden as a silent no-outcome branch. The third says that an unmatched full atom is never ignored: it either localizes to a concrete row, routes to a higher package, or makes the key cut-silent.

Theorem 11.4 (BN5-FullShadowLocalization)**.** *Every cut-active negative full residual key routes to*

$$\mathrm{CritC}, Q, E, L, X1, X2, X3, X4,$$

or to verified gain, BC, UN, HN, BUD, selector, exact route, or strict descent. Therefore, in a no-outcome branch, every negative full residual key is cut-silent.

Proof. Fix a negative full residual key κ that is cut-active after BN4 cancellation. The key contains a projected semantic signature, active-antichain code, projected transport type, and signed integer mass. Refine the negative full mass into unit full atoms and build the shadow graph

$$\mathcal{B}_\kappa = (L_\kappa, R_\kappa, E_\kappa).$$

A left vertex is a full atom that remains negative under the BN4 key, and a right vertex is a quotient candidate shadow with the same activation-exact key. An edge is admitted only when the quotient shadow preserves the projected semantic signature, active-antichain code, projected transport type, frontier order, charge owner, obligation state, origin/kernel footprint, and mode-projection record. Thus any matching is key-preserving and cut-activation preserving.

If the shadow graph has a complete matching for the negative full atoms, the matched quotient mass cancels the alleged negative full residual in the same activation-exact key, contradicting the BN4 residual ledger. Therefore a genuinely negative cut-active key must expose either an unmatched full atom or a Hall-deficient subset. This is the obligation `negativeFullResidualLocalized`.

For a Hall-deficient subset $T \subseteq L_\kappa$, the finite checker chooses the least deficient set in canonical row order and inspects the first missing shadow coordinate. A semantic impossibility routes to $X2$, a direction or transport impossibility routes to $X3$, a resource or Hall impossibility routes to $X4$ or $X1$, and mode or frontier failures route to CritC, Q, E, L. If the deficient region already contains a bounded replacement, hereditary object, budget object, packet seed, exact route, or strict descent, that higher-priority route is emitted. Hence a Hall deficit cannot remain silent; this is `hallDeficitRoutesNamedOutcome`.

If instead the first obstruction is an unmatched full atom, the atom's origin, kernel, obligation, frontier, and charge records determine a concrete localization row. A missing localization row is an X-failure; a mode-unsafe restoration is a Q or CritC route; an exposure-only mismatch is an E route; and an overlap-loss mismatch is an L route. If none of these routes is active, the unmatched atom is not cut-active and the key is cut-silent. This is `unmatchedShadowNotSilent`.

Therefore, in a no-outcome branch, no cut-active negative full residual key survives. All surviving residual keys are nonnegative quotient residual keys, and on every proper cut

$$\Omega(S, A \setminus S) = \sum_{\kappa \in Q^{res}} \gamma(\kappa)\alpha_\kappa(S) \geq 0.$$

□

11.5 Package C, renamed PkgC: separating consumers

The package formerly denoted Package C is named PkgC to avoid ambiguity with the critical kind CritC.

For a surviving quotient residual key κ, if its active minimal-consumer antichain has a disjoint pair (M, N) with at least one nonsingleton member, then PkgC builds request traces, first multi-active joins, activation-exact full-restoration universes, and quotient-to-full matching graphs. Either the quotient residual mass is fully matched, contradicting BN4 residuality, or Hall failure yields a kind Q row, an X-failure, or an unbounded route.

11.5.1 PkgC separating-consumer proof obligations

The accepted residual-band theorem records the PkgC separating-consumer step as checker-visible obligations:

```
quotientToFullMatchingKeyPreserving
separatingConsumersSingletonized
```

The first field says that every quotient-to-full restoration graph preserves the BN4 activation-exact key. The second says that, after all PkgC routes are absent, every disjoint active pair of minimal consumers for a surviving quotient residual key is singleton-singleton.

Theorem 11.5 (Separating-consumer theorem)**.** *In a minimum-rank no-outcome branch, every disjoint active pair of minimal consumers for a surviving quotient residual key consists of singleton sets.*

Proof. Let κ be a surviving active quotient residual key, and suppose that its active minimal-consumer antichain has a disjoint pair (M, N) with at least one nonsingleton member. PkgC traces each request in $M \cup N$, records the first multi-active join, and constructs a full-restoration universe for the quotient atoms participating in the pair. Restoration candidates are grouped by the same projected semantic signature, active-antichain code, projected transport type, frontier order, charge owner, obligation footprint, and origin/kernel footprint used by BN4. Thus any quotient-to-full matching is key preserving; this is `quotientToFullMatchingKeyPreserving`.

If the full-restoration graph fully matches the quotient residual mass, the matched full atoms produce a same-key cancellation or a full-shadow localization, contradicting that κ survived BN4 and BN5 as a positive active quotient residual key. If the graph does not fully match, the least Hall-deficient restoration set gives a Q row, an X-failure, a verified gain, a selector seed, an exact route, or strict residual descent. Therefore a nonsingleton disjoint active pair cannot persist in a no-outcome minimum-rank branch.

Consequently all disjoint active pairs in the minimal-consumer antichain of a surviving quotient residual key are singleton-singleton. This is `separatingConsumersSingletonized`. V54 then applies. For each surviving active quotient residual key, there exists

$$E(\kappa) = \{a : \{a\} \in \mathcal{M}_\kappa\}, \qquad |E(\kappa)| \geq 2,$$

with

$$\alpha_\kappa(S) = 1_{E(\kappa) \pitchfork S}.$$

□

11.6 BN6: hypergraph cellization and packet collapse

Group equal footprints:

$$w_E = \sum_{\kappa : E(\kappa) = E} \gamma(\kappa).$$

Then

$$\boxed{\Omega(S, A \setminus S) = \sum_{\substack{E \subseteq A \\ |E| \geq 2}} w_E 1_{E \pitchfork S}, \qquad w_E \in \mathbb{Z}_{\geq 0}.}$$

Together with the constant-cut equation

$$\Omega(S, A \setminus S) = D > 0,$$

V53 yields packet prototypes.

Theorem 11.6 (BN6-HypergraphPacket)**.** *Every BCEL-ready positive nucleus in a minimum-rank no-outcome branch produces a positive packet prototype of type*

$$Pair, \qquad BalancedTri, \qquad FullSpan.$$

Every positive packet mass carries atom witnesses.

For $|A| = 3$, the case can be mixed:

$$D = 2p + w_A.$$

Both a balanced-triple packet and a full-span packet may be present.

12 Finite rigidity and consumer antichains

Theorem 12.1 (V53 constant-cut hypergraph rigidity)**.** *Let A be finite with $|A| = q \geq 2$. Let $w_E \in \mathbb{Z}_{\geq 0}$ for every $E \subseteq A$, $|E| \geq 2$. Assume*

$$\sum_{E \pitchfork S} w_E = D > 0$$

for every nonempty proper $S \subsetneq A$. Then:

1. *if $q = 2$, then $w_A = D$;*
2. *if $q = 3$, all pair weights are equal, say p, and $w_A = D - 2p$;*
3. *if $q \geq 4$, every proper hyperedge weight vanishes and $w_A = D$.*

Proof. For $T \subseteq A$, set

$$W(T) = \sum_{E \subseteq T,\ |E| \geq 2} w_E, \qquad W = W(A).$$

A hyperedge crosses S exactly when it is not contained in S and not contained in $A \setminus S$. Hence

$$\sum_{E \pitchfork S} w_E = W - W(S) - W(A \setminus S) = D,$$

so

$$W(S) + W(A \setminus S) = K := W - D.$$

For singleton $\{i\}$, $W(A \setminus \{i\}) = K$. For pair $\{i, j\}$,

$$w_{ij} + W(A \setminus \{i, j\}) = K.$$

Subtracting gives

$$w_{ij} = \sum_{\substack{E \subseteq A \setminus \{i\} \\ j \in E, |E| \geq 2}} w_E.$$

For $q = 3$, this forces all pair weights equal. For $q \geq 4$, the right side contains at least two distinct pair terms of the same common weight, forcing that common pair weight to be zero. Then every proper hyperedge lies in such a zero sum and vanishes. The cases follow. □

Theorem 12.2 (V54 consumer-antichain normal form)**.** *Let $r : 2^A \to \{0, 1\}$ be monotone with $r(\varnothing) = 0$. Let*

$$\mathcal{M} = \min_{\subseteq} \{S \neq \varnothing : r(S) = 1\}.$$

Define

$$\kappa(S) = r(S) r(A \setminus S).$$

Then $\kappa \not\equiv 0$ iff $\mathcal{M}$ contains two disjoint members. If every disjoint pair in $\mathcal{M}$ consists of singleton sets, then κ is exactly the cut indicator of

$$E = \{a : \{a\} \in \mathcal{M}\}.$$

13 Packet extraction and selector coverage

13.1 Selector universe

The final selector universe is

$$\boxed{\mathcal{K}(C) = K_2(C) \cup K_3(C) \cup K_{sp}(C).}$$

The pair selector family is

$$K_2(C) = \{(pair, h_1, h_2, \theta_2) : h_1, h_2 \in H(C), \theta_2 \in \Theta_2(|C|)\}.$$

The triple selector family is

$$K_3(C) = \{(tri, h_1, h_2, h_3, \theta_3) : h_i \in H(C), \theta_3 \in \Theta_3(|C|)\}.$$

The spine selector family is

$$K_{sp}(C) = \{(sp, h_{in}, h_{cut}, h_{out}, \theta_{sp}) : h_* \in H(C), \theta_{sp} \in \Theta_{sp}(|C|)\}.$$

The handle bound and payload bounds give

$$|\mathcal{K}(C)| \le |C|^3 (\log |C|)^{O(1)}.$$

13.2 Pair packet extraction

A pair packet

$$Pair(a, b; M, Cell(\{a, b\}))$$

with $M > 0$ has an atom witness $u \in Cell(\{a, b\})$. Its payload is

$$\theta_2(u) = (\bar{\sigma}(u), \tau(u), \mathcal{F}(u), \mathsf{Ch}(u), Obl(u), Orig(u), Ker(u), Pref(u), Dir(u), Bud(u), \mathrm{ActCode}(u), ranktag(u)).$$

Then

$$K_u = (pair, h(a), h(b), \theta_2(u)) \in K_2(C).$$

Theorem 13.1 (Pair packet seed)**.** *Every positive pair packet yields a faithful K_2 selector seed, or routes a first colour, frontier, charge, obligation, activation, direction, budget, rank, exact-route, or descent obstruction.*

13.3 Balanced-triple extraction

A balanced triple gives nonempty equal-mass cells

$$Cell(ab), \quad Cell(ac), \quad Cell(bc).$$

A compatible triad

$$(u_{ab}, u_{ac}, u_{bc})$$

must satisfy endpoint consistency, projected signature coherence, frontier coherence, charge coherence, obligation coherence, direction coherence, budget coherence, activation coherence, and rank coherence.

Theorem 13.2 (Balanced-triple seed)**.** *Every positive balanced-triple packet yields a faithful K_3 selector seed, or routes the first triad mismatch. A single compatible triad is enough; a perfect matching of all triple mass is not required.*

13.4 Full-span spine extraction

A full-span packet

$$FullSpan(A; M, Cell(A))$$

with $M > 0$ has atom witnesses with footprint A. For a least witness u, form the trace support

$$\mathcal{S}(u) = \mathrm{Comp}(\mathrm{Sat}_C(Dep^-(u) \cup \bigcup_{a \in A} ReqTr(u, a))).$$

Its primitive incidence graph $\Gamma(u)$ must either be a clean spine or route.

A clean full-span spine is connected, path-like after suppressing neutral degree-two profile-only vertices, covers exactly the full-span footprint, has accepted unary transitions, frontier-faithful prefixes, full-mode obligation cleanliness, unique charge ownership, budget within envelope, no unresolved hereditary side closure, no lower packet, and proper decoded support.

It compresses into three handles:

$$(h_{in}, h_{cut}, h_{out})$$

and a finite payload θ_{sp}. The deterministic spine walker reconstructs the path inside C.

Theorem 13.3 (Full-span spine seed)**.** *Every positive full-span packet yields a faithful K_{sp} selector seed, or routes to* $\mathrm{CritC}, Q, E, L, X$*, verified gain, BC, UN, HN, BUD, selector descent, exact route, or strict residual descent.*

Theorem 13.4 (BCEL seed, patched)**.** *If C is normalized, $\Lambda(C) > 0$, and no verified gain, exact route, named outcome, selector descent, strict residual descent, HN-active object, or budget-active object survives, then*

$$\exists K \in K_2(C) \cup K_3(C) \cup K_{sp}(C), \quad \exists P, \quad \mathrm{Faithful}(C, K, P).$$

14 Package R: selector realization and strict surplus

14.1 Public realizer contract

For every selector

$$K \in K_2(C) \cup K_3(C) \cup K_{sp}(C),$$

the public realizer output type is

$$\mathrm{Real}(C, K) \in \{\bot, Gain(U, R, \Pi)\}.$$

A gain is returned only after

$$\mathrm{VerifyDW}_{\mathrm{exp}}(C, U, R, \Pi) = accept.$$

A $\bot$-certificate for a faithful selector is one of:

$$BotHN, \qquad BotBUD, \qquad BotSeed.$$

The static reject case is impossible for faithful selectors.

Theorem 14.1 (Selector realization)**.** *For every selector K,*

$$\mathrm{Real}(C, K) = Gain(U, R, \Pi) \Longrightarrow \mathrm{VerifyDW}_{\mathrm{exp}}(C, U, R, \Pi) = accept.$$

Moreover, if

$$\mathrm{Faithful}(C, K, P)$$

and

$$\mathrm{Real}(C, K) = \bot,$$

then

$$HNActive_{\leq P}(C) \vee BudgetActive_{\leq P}(C) \vee \exists K', P' \, [\mathrm{Faithful}(C, K', P') \wedge P' \prec P].$$

14.2 Strict charge surplus

A selector blueprint has a charge-surplus injection if there is an injective map

$$\psi_K : \mathsf{Ch}(R_K) \hookrightarrow \mathsf{Ch}(U_K)$$

preserving weights, type, truth vectors, relation tables, frontier, origin, kernel, obligation, prefix, direction, saturation, budget, charge, source incidence, consumer incidence, and dependency edges, with at least one positive-weight unmatched support charge.

Theorem 14.2 (R-ChargeSurplus)**.** *If*

$$\text{Faithful}(C,K,P), \qquad NoHN_{\leq P}(C), \qquad NoBUD_{\leq P}(C), \qquad NoFaithful_{<P}(C),$$

then the decoded selector blueprint admits a charge-surplus injection. Consequently,

$$|R_K| \leq |U_K| - 1,$$

and a $\text{VerifyDW}_{\text{exp}}$*-accepted gain is produced.*

Pair selectors fold two active traces into one, leaving one residual atom unmatched. Balanced triples fold three compatible edge traces into two selector-core traces. Spines compress a clean path and omit at least one interior transition charge unless HN-active, budget-active, or lower-rank packet structure is present.

15 Simultaneous HN-BUD rank-negative closure

Let $(\mathcal{P}_C, \prec)$ be the finite packet-rank set. A rank has the form

$$P = (kind, span, massclass, colourclass, frontierclass, chargeclass, \\ budgetclass, hereditaryclass, activationclass, ranktag, code).$$

Define

$$NoFaithful_{<P}(C) := \neg\exists K, Q\,[K \in \mathcal{K}(C) \wedge Q \prec P \wedge \text{Faithful}(C,K,Q)].$$

A hereditary active certificate L has rank $\rho_H(L)$. A budget active certificate B has rank $\rho_B(B)$. The combined blocker graph has H vertices and B vertices. H2 sidecar entries give edges

$$H(L) \to B(B),$$

and B2 entries give edges

$$B(B) \to H(L).$$

Every edge satisfies

$$\rho(dst) \leq \rho(src),$$

and if ranks are equal,

$$\beta(dst) < \beta(src).$$

Thus the graph is acyclic.

Theorem 15.1 (HB negative closure)**.** *For every packet rank* P*,*

$$NoHereditary(C,\mathcal{N}_H) \wedge NoBudget(C,\mathcal{N}_B) \wedge NoFaithful_{<P}(C) \Longrightarrow NH_{\leq P}(C) \wedge NB_{\leq P}(C).$$

Proof. Assume an active HN or BUD vertex of rank at most P, and choose one minimizing (ρ,β). The local sidecar soundness rule forces a blocker edge to an active vertex of strictly smaller (ρ,β), contradiction. □

Theorem 15.2 (Selector-silence induction)**.** *If selector silence, NoHereditary, and NoBudget sidecars are recorded, and no lower branch is active, then for every rank* P*,*

$$NoFaithful_{\leq P}(C), \qquad NH_{\leq P}(C) \wedge NB_{\leq P}(C).$$

16 Oracle, ZeroSlack, and residual-band minimization

16.1 Rank-ordered PCCOracle

The oracle enumerates selectors in packet-rank order.

```
PCCOracle_exp(C):
    h = HResolve_exp(C)
    if h is Minimum(C,M,Mc): return Minimum(C,M,Mc)
    if h is Gain(C,U,R,Pi): return Gain(C,U,R,Pi)
    store NoHereditary(C,N_H)

    b = BudgetResolve_exp(C)
    if b is Minimum(C,M,Mc): return Minimum(C,M,Mc)
    if b is Gain(C,U,R,Pi): return Gain(C,U,R,Pi)
    store NoBudget(C,N_B)

    construct finite rank list P_1,...,P_M
    for j = 1 to M:
        for K in K_{P_j}(C):
            out = Real(C,K)
            if out is Gain(C,U,R,Pi): return Gain(C,U,R,Pi)
            store Real(C,K)=bot
        store SelectorSilent_{<=P_j}(C)

    return ZeroSlack(C,Z)
```

The ZeroSlack certificate stores normalized-input record, NoHereditary sidecar, NoBudget sidecar, HB blocker graph, rank list, selector silence ledger for K_2, K_3, K_{sp}, no-lower ledger, no-faithful rank induction, $NH_{\le P} \wedge NB_{\le P}$ rank induction, and BCEL contradiction proof.

16.1.1 ZeroSlack proof obligations

The accepted package-sufficiency theorem records the residual-band exact-minimization block as a first-class theorem object, ResidualBandExactMinimization. Its boundary is

$\Lambda(C_0) \le O(\log |C_0|) \implies \mathrm{PCCMin}_{\exp}(C_0)$ returns an exact minimum equivalent circuit in polynomial time.

The checker-visible ZeroSlack obligations are the following record fields:

```
pccMinReturnsExactMinimum
residualSlackBounded
zeroSlackSound
terminalCarrierPreservesSemantics
terminalizationSizePreserving
closedFullWordRealizesCircuit
quotientEqualityNotConstructive
wholeSpanCheaperImpliesStrictDescent
transparentSaturationCostBalanced
interfaceExposureRoutesToE
originKernelObligationClosureRouted
projectionPositivityNotLostSilently
firstNontransparentStepRecorded
zeroSlackEarlierRoutesExcluded
positiveResidualWitnessYieldsBCELReady
positiveResidualWitnessExists
finiteAnchorSetExtracted
booleanAnchorAlgebraOrRoute
minimalPositiveNucleus
properCutConstantEquation
anchorSizeAtLeastTwo
bcelAnchorAlgebraBooleanOrRoutes
sideTightOnlyNoOverclaim
fourCornerOptimaCarrierCompatible
sideTightCompletionExists
tightBasisValueEqualsDelta
requestPredicatesStable
minimalConsumerAntichainsExact
jointSideTightRealizability
runtimeIntegersSeparatedFromFiniteState
```

```
incidenceAtomsAccountedExactlyOnce
activationByActiveAntichain
activationEqualityWithoutCutEnumeration
sameKeyCancellationExact
noOppositeSignSameKeyResidual
integerMassLedgerExact
negativeFullResidualLocalized
hallDeficitRoutesNamedOutcome
quotientToFullMatchingKeyPreserving
separatingConsumersSingletonized
unmatchedShadowNotSilent
selectorUniverseCompleteForPackets
realizerBotOnlyHNBUDBlockedOrLowerRank
hbBlockerGraphAcyclic
zeroSlackContradictionFromPositiveSlack
zeroSlackCertificatePolynomialSize
```

Mathematically, these fields assert that the gain loop is residual-band bounded, that a ZeroSlack output is exact, that terminal MuBridge translates between whole circuits and closed full words without semantic or size drift, that quotient equality is nonconstructive unless lifted, that whole-span cheaper words give strict residual descent, that transparent saturation preserves positivity, that nontransparent saturation and interface exposure route to named outcomes, that earlier gain/exact/named routes have been positively excluded, that positive residual slack yields a BCEL-ready positive nucleus unless an earlier route fires, that a finite anchor set is extracted, that Boolean anchor algebra failures route rather than remain silent, that a minimal positive nucleus with at least two anchors has a constant proper-cut equation, that BN2 uses only carrier-compatible side-tight optima and never overclaims from non-tight coherent bases, that BN3 builds stable finite request predicates, exact minimal-consumer antichains, jointly realizable side-tight envelopes, arithmetic-separated runtime integers, and exact incidence ledgers, that BN4 represents activation by active antichains, proves equality without cut enumeration, cancels same-key positive and negative masses exactly, leaves no opposite-sign same-key residual, and checks the integer mass ledger, that BN5 localizes negative full residuals, routes Hall deficits, and never leaves unmatched shadows silent, that PkgC preserves activation-exact keys in quotient-to-full matching and singletonizes separating consumers, that the finite selector universe covers all positive packet prototypes, that realizer failure is only by typed HN/BUD or lower-rank blocking, that the blocker graph is acyclic, that positive slack contradicts the ZeroSlack ledger, and that the certificate has polynomial size.

Theorem 16.1 (Rank-parametric ZeroSlack)**.** *If*

$$\mathrm{PCCOracle}_{\exp}(C) = \mathrm{ZeroSlack}(C, \mathcal{Z}),$$

then

$$\Lambda(C) = 0.$$

Proof. Assume for contradiction that $\Lambda(C) > 0$ and that the oracle nevertheless returns $\mathrm{ZeroSlack}(C, \mathcal{Z})$. The certificate first rules out higher-priority exits: HResolve did not return a minimum or a gain, BudgetResolve did not return a minimum or a gain, and the no-lower ledger excludes named obstructions, exact routes, selector descents, and strict residual descents. Hence any positive residual witness must survive into the residual-witness stack.

The residual-witness obligations then give a saturated projection-positive BCEL-ready hull $BCELReady(C, H, A, \mathcal{X}, D)$ with $D > 0$, unless one of the excluded routes has already fired. The anchor-algebra obligation says that every failure of Boolean anchor algebra, saturation compatibility, or proper-cut constancy is routed to a named outcome, selector, exact route, gain, or strict descent. Since the ZeroSlack branch asserts those routes are absent, the positive nucleus satisfies the BCEL hypotheses.

BN2 through BN6 convert this constant-cut positive nucleus into a positive pair, balanced-triple, or full-span packet. Selector completeness says every such packet has a representative in the finite selector universe $K_2(C) \cup K_3(C) \cup K_{sp}(C)$, unless a named route or verified gain has already fired. Therefore positive residual slack yields a faithful selector (K, P) at some finite packet rank.

Choose a faithful selector of least rank. If its realizer emits a gain, the oracle would return that gain before ZeroSlack. If the realizer emits $\perp$, the realizer obligation permits only an HN blocker, a BUD blocker, or a lower-rank faithful-selector blocker. The lower-rank case contradicts minimality and selector silence. The HN/BUD cases contradict the simultaneous HB negative closure, because the blocker graph is acyclic by the recorded decreasing rank and tie-break measure. Thus the faithful selector cannot be blocked.

So positive residual slack forces either an earlier route, a realized gain, or an unblocked faithful selector. All are incompatible with the emitted ZeroSlack ledger. Therefore $\Lambda(C) > 0$ is impossible, and $\Lambda(C) = 0$. The certificate-size obligation ensures the rank list, selector-silence records, sidecars, blocker graph, and BCEL contradiction proof are polynomially bounded under the accepted schedule. □

16.2 PCCMin

```
PCCMin_exp(C0):
    C = C0
    while True:
        n = NormalizeOrGain_exp(C)
        if n is Gain(C,U,R,Pi):
            C = C[U <- R]
            continue
        if n is Normal(Cnu):
            C = Cnu

        out = PCCOracle_exp(C)
        if out is Minimum(C,M,Mc): return M
        if out is ZeroSlack(C,Z): return C
        if out is Gain(C,U,R,Pi):
            C = C[U <- R]
```

Every nonterminal gain step lowers Λ by at least one. Hence the number of gain steps is at most $\Lambda(C_0)$.

Theorem 16.2 (Residual-band exact minimization)**.** *If the accepted package is valid and*

$$\Lambda(C_0) \le O(\log |C_0|),$$

then $\mathrm{PCCMin}_{\mathrm{exp}}(C_0)$ *returns an exact minimum equivalent circuit in polynomial time.*

Proof. The proof is the checker-visible obligation `pccMinReturnsExactMinimum`. Each nonterminal gain returned by NormalizeOrGain or by $\mathrm{PCCOracle}_{\mathrm{exp}}$ is accepted by $\mathrm{VerifyDW}_{\mathrm{exp}}$, so the global slack law decreases Λ by at least one. The field `residualSlackBounded` records that the starting residual band is $O(\log |C_0|)$, so only polynomially many gain iterations can occur. When $\mathrm{PCCOracle}_{\mathrm{exp}}$ returns $\mathrm{Minimum}(C, M, M_c)$, the hereditary or budget exact route supplies an exact minimum witness. When it returns $\mathrm{ZeroSlack}(C, \mathcal{Z})$, Theorem 16.1 and `zeroSlackSound` give $\Lambda(C) = 0$, hence $|C| = \mu(C)$ and the current circuit is already exact minimum. Therefore the algorithm terminates after polynomially many checked steps and returns an exact minimum equivalent circuit. □

17 Locked NAND SAT embedding

17.1 Carrier slots and invariants

The locked NAND builder uses tagged disjoint carrier families

$$X \sqcup T \sqcup O \sqcup R \sqcup L \sqcup \{z\}.$$

Here X are primary inputs, $T = \{t_j\}$ are trace value slots, $O = \{o_e\}$ are source occurrence slots, $R = \{r_e\}$ are source locks, $L = \{\ell_j\}$ are trace locks, and z is the final fresh lock.

The invariant *G-Sep*$^+$ requires pairwise distinct inputs inside equality and trace macros, fresh occurrence slots for duplicate NAND sources, constants enforced by M_0, M_1 rather than carrier constants, globally fresh macro locks, final lock z appearing only in the final output construction, and prefix-conjunction nodes with distinct underlying check-lock sets.

The invariant *G-Coh* gives a coherence witness assigning all distinguished checks the value 1.

17.2 Macros

The equality macro $M_=$ on inputs (r, u, s) has ten gates:

$$\begin{aligned} &a_1 = \mathsf{N}(r, u), \quad a_2 = \mathsf{N}(r, s), \quad a_3 = \mathsf{N}(a_1, a_1), \quad a_4 = \mathsf{N}(a_3, s),\\ &a_5 = \mathsf{N}(r, a_1), \quad a_6 = \mathsf{N}(a_5, a_5), \quad a_7 = \mathsf{N}(a_6, a_2), \quad a_8 = \mathsf{N}(a_4, a_7),\\ &a_9 = \mathsf{N}(r, r), \quad a_{10} = \mathsf{N}(a_9, u). \end{aligned}$$

The distinguished output is

$$a_8 = r[(u \wedge s) \vee (\neg u \wedge \neg s)].$$

All ten outputs are exposed.

The constant-one macro is

$$b_1 = \mathsf{N}(r, u), \qquad b_2 = \mathsf{N}(b_1, b_1) = r \wedge u.$$

The constant-zero macro is

$$d_1 = \mathsf{N}(r, u), \quad d_2 = \mathsf{N}(r, d_1), \quad d_3 = \mathsf{N}(d_2, d_2) = r \wedge \neg u.$$

The NAND trace macro M_N on inputs (ℓ, t, u, v) has eighteen gates:

$$\begin{aligned} &q_1 = \mathsf{N}(\ell, t), \quad q_2 = \mathsf{N}(q_1, q_1), \quad q_3 = \mathsf{N}(\ell, u), \quad q_4 = \mathsf{N}(\ell, v),\\ &q_5 = \mathsf{N}(q_2, q_3), \quad q_6 = \mathsf{N}(q_2, u), \quad q_7 = \mathsf{N}(q_6, q_6), \quad q_8 = \mathsf{N}(q_7, q_4),\\ &q_9 = \mathsf{N}(\ell, q_1), \quad q_{10} = \mathsf{N}(q_9, q_9), \quad q_{11} = \mathsf{N}(q_{10}, u), \quad q_{12} = \mathsf{N}(q_{11}, q_{11}),\\ &q_{13} = \mathsf{N}(q_{12}, v), \quad q_{14} = \mathsf{N}(q_5, q_8), \quad q_{15} = \mathsf{N}(q_{14}, q_{14}), \quad q_{16} = \mathsf{N}(q_{15}, q_{13}),\\ &q_{17} = \mathsf{N}(\ell, \ell), \quad q_{18} = \mathsf{N}(q_{17}, t). \end{aligned}$$

The distinguished output is

$$q_{16} = \ell[t = \mathsf{N}(u, v)].$$

All eighteen outputs are exposed.

Prefix conjunction of two check bits uses two gates and exposes p and $\neg p$. The final ternary conjunction uses four gates and exposes only

$$F_\varphi = z \wedge T_\varphi \wedge y_{out},$$

where z is a fresh final lock.

17.3 Baseline and threshold

For a NAND circuit φ with m gates and source occurrence counts $w_=, w_0, w_1$, the baseline is

$$B_\varphi^{NAND} = 18m + 10w_= + 3w_0 + 2w_1 + 2(3m - 1).$$

The full word has

$$|W_\varphi^{NAND}| = B_\varphi^{NAND} + 4.$$

Theorem 17.1 (BaselineDistinct)**.** *The baseline tuple contains exactly B_φ^{NAND} pairwise distinct nonconstant nonprojection functions. Hence every direct-wire realization of the baseline tuple has at least B_φ^{NAND} NAND gates, and the constructed baseline is exact.*

Theorem 17.2 (Locked NAND threshold)**.** *For every NAND circuit φ,*

$$\varphi \notin SAT \Longrightarrow \mu(W_\varphi^{NAND}) = B_\varphi^{NAND},$$

$$\varphi \in SAT \Longrightarrow B_\varphi^{NAND} + 1 \le \mu(W_\varphi^{NAND}) \le B_\varphi^{NAND} + 4.$$

Consequently,

$$\varphi \in SAT \Longleftrightarrow \mu(W_\varphi^{NAND}) > B_\varphi^{NAND}, \qquad \Lambda(W_\varphi^{NAND}) \le 4.$$

17.4 Output convention and threshold proof details

The locked NAND threshold uses the direct-wire multi-output convention of Section 2.1. An output coordinate may be wired to a boundary coordinate, to a carrier constant, or to the output of a NAND gate. Repeating an already available output coordinate does not add a NAND gate. This convention concerns only the external output tuple. It does not make checked data constants free inside the locked construction: constants that participate in trace constraints are still enforced by the macros M_0 and M_1 as required by G-Sep+.

Lemma 17.3 (DirectWireOutputLowerBound)**.** *Let $F = (f_1, \ldots, f_B)$ be a direct-wire output tuple over a fixed carrier. If the functions f_i are pairwise distinct and each f_i is nonconstant and nonprojection, then every NAND direct-wire realization of F has at least B NAND gates.*

Proof. An output coordinate that is a boundary coordinate is a projection, and an output coordinate wired to a carrier constant is constant. Hence every nonconstant nonprojection output coordinate must be the output of some NAND gate. One NAND gate output computes one Boolean function. Since the f_i are pairwise distinct, no single gate output can serve two of them. Thus the realization contains at least B distinct NAND gate outputs, hence at least B NAND gates. □

Lemma 17.4 (MacroDistinct)**.** *Under G-Sep+, every exposed baseline output from an equality, constant, trace, or prefix macro is tagged by its macro instance and fresh lock or occurrence slot. The exposed baseline functions are pairwise distinct, nonconstant, and nonprojection. Therefore the baseline tuple forces exactly B_φ^{NAND} NAND gates.*

Proof. The accepted G certificate separates this lemma into the following explicit obligations: macro truth signatures are pairwise distinct, macro outputs are nonconstant, macro outputs are nonprojections, cross-instance copies are carrier-tagged, prefix outputs are separated from macro outputs, duplicate source occurrences use fresh slots, and the projection boundary is the positive boundary-coordinate convention of the direct-wire model.

For a single equality, constant, or NAND-trace macro instance, Appendix A gives the complete output truth signatures and the positive projection signatures in the declared variable order. Direct comparison of those finite bit strings shows that the exposed outputs of the instance are pairwise distinct and nonconstant. It also shows that no exposed output equals any positive projection of an input coordinate. This is the relevant projection test for the direct-wire convention: a boundary wire supplies a positive boundary coordinate, while negations and other derived functions require NAND gates unless they are carrier constants or already available outputs.

Now consider two different macro instances. By G-Sep+, their private locks and occurrence variables are renamed into disjoint carrier coordinates. If two same-shape local signatures are copied into different instances, the resulting global functions depend on different carrier coordinates. Assign the first instance's active lock and vary the local data coordinate on which its output essentially depends while holding the second instance's carrier fixed; the first output changes and the second does not. Hence cross-instance copies are distinct global functions. Fresh occurrence slots for duplicate NAND sources are needed here: two occurrences of the same source value may be semantically equal on coherent traces, but they are distinct carrier functions before the trace constraints are imposed, so their exposed macro outputs do not collapse in the baseline tuple.

The prefix-conjunction outputs are separated for the same reason. Each prefix node uses its own check-lock set and depends essentially on a prefix check coordinate not used by any equality, constant, or NAND-trace macro output. G-Sep+ gives distinct check-lock sets for different prefix nodes and keeps them disjoint from macro locks. Therefore a prefix output cannot equal a macro output or another prefix output unless the two tagged carrier supports are identical, which the construction forbids. The exposed negated prefix output $\neg p$ is also not a free projection: it is a NAND-derived function over the prefix node's local carrier and is counted as an exposed baseline output.

Thus the whole baseline tuple contains exactly the displayed number of pairwise distinct nonconstant nonprojection functions: $18m$ trace-macro outputs, $10w_=$ equality outputs, $3w_0$ constant-zero outputs, $2w_1$ constant-one outputs, and $2(3m-1)$ prefix outputs. Applying DirectWireOutputLowerBound gives $\mu(\text{baseline}) \geq B_\varphi^{NAND}$. The constructed baseline exposes exactly those B_φ^{NAND} NAND outputs, so the lower bound is tight and the baseline is exact. □

Lemma 17.5 (TraceEquivalence)**.** *Let T_φ be the final prefix conjunction of all distinguished equality, constant, and NAND trace checks. For every assignment to the primary inputs X, there exists an extension to the trace, occurrence, and lock slots with $T_\varphi = 1$ and $y_{out} = 1$ if and only if $\varphi(X) = 1$. Consequently, $T_\varphi \wedge y_{out}$ is satisfiable if and only if φ is satisfiable.*

Proof. The proof uses the following trace obligations, each of which is part of the accepted G certificate: active locks are forced by distinguished outputs; equality locks force equality of source and occurrence slots; constant locks force the occurrence value to be the stated constant; NAND trace locks force gate equations; the prefix covers all distinguished checks; the induction follows the topological order of the NAND circuit; the output slot y_{out} is the trace slot of the declared circuit output; and duplicate source occurrences are assigned fresh occurrence slots.

For the forward direction, suppose $\varphi(X) = 1$. Evaluate the NAND circuit in topological order. Assign t_j to the value of gate j, assign every source occurrence slot to the value of the source it names, and set every equality, constant, and trace lock to its active value. For an input or earlier-gate source, the corresponding $M_=$ distinguished output is then

$$r[(u \wedge s) \vee (\neg u \wedge \neg s)] = 1,$$

because the occurrence value u and source value s agree. For a constant-one occurrence, the M_1 distinguished output is $r \wedge u = 1$; for a constant-zero occurrence, the M_0 distinguished output is $r \wedge \neg u = 1$. For a NAND gate with source values u, v and trace value t, the M_N distinguished output is

$$\ell[t = \mathsf{N}(u, v)] = 1.$$

The prefix-conjunction tree is exact and includes each distinguished check once, so $T_\varphi = 1$. Since $\varphi(X) = 1$, the trace slot corresponding to the declared output gate has value 1, hence $y_{out} = 1$.

For the reverse direction, suppose there is an extension with $T_\varphi = 1$ and $y_{out} = 1$. Exactness and coverage of the prefix tree imply that every distinguished check is one. For each equality check, the equation

$$r[(u \wedge s) \vee (\neg u \wedge \neg s)] = 1$$

forces $r = 1$ and $u = s$. For constant checks, $r \wedge u = 1$ forces $u = 1$, and $r \wedge \neg u = 1$ forces $u = 0$. For each NAND trace check,

$$\ell[t = \mathsf{N}(u, v)] = 1$$

forces $\ell = 1$ and $t = \mathsf{N}(u, v)$. Fresh occurrence slots ensure that duplicate source occurrences are separately constrained and cannot be silently identified. By induction over the topological order of the NAND DAG, every trace slot therefore equals the value obtained by evaluating the corresponding gate of φ on X: the base cases are primary inputs and constant occurrences, and the induction step is the forced NAND trace equation for each gate. The distinguished output slot y_{out} is, by construction, the trace slot of the declared circuit output. Since $y_{out} = 1$, the evaluated circuit output is one, so $\varphi(X) = 1$. □

Lemma 17.6 (ZeroOutputConvention)**.** *If $\varphi \notin SAT$, then the final output $F_\varphi = z \wedge T_\varphi \wedge y_{out}$ is the identically zero function. Appending this zero output coordinate to the baseline tuple does not increase the minimum number of NAND gates, so $\mu(W_\varphi^{NAND}) = B_\varphi^{NAND}$ in the unsatisfiable case.*

Proof. The accepted G certificate separates the zero-output step into explicit obligations: unsatisfiability makes the trace predicate $T_\varphi \wedge y_{out}$ identically false; the final coordinate is therefore identically zero; a zero output coordinate may be wired to a carrier constant at no NAND cost; repeated output coordinates add no NAND gate; and the direct-wire multi-output size measure counts NAND gates only, not the number of external output coordinates.

Assume $\varphi \notin SAT$. If some assignment to the full carrier made $T_\varphi \wedge y_{out} = 1$, then its restriction to the primary inputs together with the assigned trace, occurrence, and lock slots would be an extension witnessing $T_\varphi = 1$ and $y_{out} = 1$. By TraceEquivalence this would imply $\varphi(X) = 1$ for that primary input assignment, contradicting unsatisfiability. Hence $T_\varphi \wedge y_{out}$ is the zero Boolean function on the whole carrier, not merely on coherent traces. Therefore

$$F_\varphi = z \wedge T_\varphi \wedge y_{out} \equiv 0.$$

Let W_B be the displayed baseline realization with exactly B_φ^{NAND} NAND gates. Form a realization of the full output tuple by using the same gates as W_B for every baseline coordinate and wiring the additional final output coordinate to the carrier constant 0. This is legal in the direct-wire output convention of Section 2.1: output coordinates may be boundary coordinates, carrier constants, or earlier gate outputs; repeated coordinates and constant coordinates contribute no additional gate. Thus

$$\mu(W_\varphi^{NAND}) \leq B_\varphi^{NAND}.$$

This convention does not make checked internal constants free. Constant source occurrences inside the trace are still certified by the locked macros M_0 and M_1; only the external output coordinate that has become the constant-zero function is wired to the carrier zero at no NAND cost.

The full word still contains the whole baseline tuple as output coordinates. MacroDistinct and DirectWire-OutputLowerBound therefore force at least B_φ^{NAND} NAND gates for any realization of the full tuple, because the baseline coordinates are pairwise distinct nonconstant nonprojections. The appended zero coordinate is constant and contributes no new lower-bound obligation. Hence

$$\mu(W_\varphi^{NAND}) = B_\varphi^{NAND}.$$

□

Lemma 17.7 (FinalLockSeparation)**.** *If $\varphi \in SAT$, then the final output $F_\varphi = z \wedge T_\varphi \wedge y_{out}$ is nonconstant, nonprojection, and distinct from every baseline output. Hence every realization of the full word needs at least one NAND gate beyond the baseline.*

Proof. The accepted G certificate separates this step into explicit obligations: a satisfying assignment makes both T_φ and y_{out} equal to one; the final lock z is fresh and appears only in the final output construction; the final output depends essentially on z; no baseline output mentions z; the final output is nonconstant and nonprojection; it is distinct from every baseline output; and this distinct final-lock output forces one extra NAND gate in the satisfiable case.

Assume $\varphi \in SAT$. By TraceEquivalence, there is an assignment to the primary inputs and an extension to trace, occurrence, and lock slots with $T_\varphi = 1$ and $y_{out} = 1$. Fix all carrier coordinates of this extension except the fresh final lock z. Then

$$F_\varphi(0, T_\varphi = 1, y_{out} = 1) = 0, \qquad F_\varphi(1, T_\varphi = 1, y_{out} = 1) = 1.$$

Thus F_φ depends essentially on z and is nonconstant.

The same formula is not a projection. It is not the positive projection z, because with $z = 1$ and either $T_\varphi = 0$ or $y_{out} = 0$, the value of F_φ is still zero. It is not the projection T_φ or y_{out}, because setting $z = 0$ makes $F_\varphi = 0$ even when that coordinate is one. It is not a projection onto any unrelated carrier coordinate because changing z as above changes F_φ while that coordinate is held fixed. The direct-wire projection boundary is positive boundary coordinates; negations or other derived functions are not free projections.

By G-Sep+, z is the unique final-lock slot and appears only in the final four-gate construction. Every baseline output is built from equality, constant, trace, and prefix carrier coordinates and therefore is independent of z. Since F_φ changes when z changes with all other carrier coordinates fixed, while every baseline output is unchanged under that variation, no baseline output can equal F_φ as a Boolean function on the full carrier.

The full word in the satisfiable case therefore contains the exact baseline tuple plus one additional Boolean function that is nonconstant, nonprojection, and distinct from all baseline coordinates. DirectWireOutput-LowerBound applied to this enlarged tuple gives

$$\mu(W_\varphi^{NAND}) \geq B_\varphi^{NAND} + 1.$$

This is the required satisfiable-case lower bound. □

Proof of Theorem 17.2. MacroDistinct gives $\mu(\text{baseline}) = B_\varphi^{NAND}$. If $\varphi \notin SAT$, ZeroOutputConvention shows that the appended final coordinate is zero-cost, so $\mu(W_\varphi^{NAND}) = B_\varphi^{NAND}$. If $\varphi \in SAT$, FinalLockSeparation gives the lower bound $\mu(W_\varphi^{NAND}) \geq B_\varphi^{NAND} + 1$, while the explicit construction realizes the baseline and the final ternary conjunction $z \wedge T_\varphi \wedge y_{out}$ using four additional gates, giving $\mu(W_\varphi^{NAND}) \leq B_\varphi^{NAND} + 4$. Since $|W_\varphi^{NAND}| = B_\varphi^{NAND} + 4$, the residual slack is at most four, and the threshold equivalence follows. □

18 Proof kernels, package manifest, and final theorem

18.1 PCC-K and PCC-K Sigma

Package outputs compile to finite kernel judgments:

$$Gain(C, U, R, \Pi), \quad Norm(C, C^\nu, T^\nu), \quad ExactMin(C, M),$$

$$ZeroSlack(C, \mathcal{Z}), \quad NoBudget(C, \mathcal{N}_B), \quad NoHereditary(C, \mathcal{N}_H),$$

$$SelectorSilent(C, \mathcal{S}_K), \quad FaithfulSeed(C, K, P).$$

Gain can arise only through $\mathrm{VerifyDW}_{\mathrm{exp}}$. ExactMin can arise only from HResolve, BudgetResolve, or ZeroSlack. ZeroSlack is a contradiction derivation from negative sidecars, BCEL seed extraction, Package R, and HB rank induction.

PCC-K Sigma represents universal package theorems schematically with finite side-condition language and fixed induction principles: DAG induction, ledger induction, obligation topological induction, trace induction, finite exhaustion, dynamic-programming induction, Hall rule, rank induction, minimal-counterexample rule, integer arithmetic, and finite hypergraph rigidity.

18.2 Interface dictionary and package manifest

The shared IfaceDict owns public constructors, route tokens, ranks, activation codes, selector families, sidecar edges, checker predicates, forbidden executable symbols, and import labels. Public output constructors are

$$\mathit{Gain},\quad \mathit{Minimum},\quad \mathit{ZeroSlack},\quad \mathit{NoBudget},\quad \mathit{NoHereditary},\quad \mathit{SelectorSilent},\quad \mathit{Faithful},\quad \mathit{Token}.$$

Critical kind CritC and package PkgC are distinct names.

The final package record is

$$\begin{aligned} P = (&P_{Base}, P_{IfaceDict}, P_{NF}, P_{Iface}, P_{Arithmetic}, P_{TruthVec}, P_{Charge}, P_{Mode}, P_E, P_N, P_{FT}, P_X,\\ &P_{Splice}, P_{BC}, P_{UN}, P_{Route}, P_{HNShape}, P_{HN}, P_{HR}, P_{BUD}, P_{NOR}, P_{FFLoss}, P_{FF},\\ &P_{RWMu}, P_{RWSpan}, P_{RWSat}, P_{RWRank}, P_{RWBCEL}, P_{CutDomain}, P_{BN2}, P_{BN3}, P_{BN4},\\ &P_{BN5}, P_C, P_{BN6}, P_{Act}, P_{Rank}, P_{Packet}, P_{K2}, P_R, P_{HB}, P_O, P_G, P_{Final}, P_K, P_\Sigma, P_{PACK}). \end{aligned}$$

The refined package order is

$$\begin{aligned} &\mathit{Kernels}, \mathit{Base}, \mathit{Iface}, \mathit{Arithmetic}, \mathit{TruthVec}, \mathit{Charge}, \mathit{Mode}\\ &\prec E \prec N \prec FT \prec FTX, X, \mathit{Splice}\\ &\prec \mathit{HNShape}, BC, UN \prec \mathit{Route} \prec \mathit{HNFull}\\ &\prec \mathit{HResolve} \prec BUD \prec NORFF \prec RW\\ &\prec BN2 \prec BN3 \prec BN4 \prec BN5 \prec \mathit{PkgC} \prec BN6\\ &\prec \mathit{Packet} \prec BCEL \prec R \prec HB \prec O \prec G \prec \mathit{Final} \prec \mathit{PACK}. \end{aligned}$$

Forbidden edges include

$$BC \leftrightarrow UN,\quad BCEL \to R,\quad BUD \to R,\quad O \to G,\quad G \to O.$$

18.3 CheckPack

$\mathrm{CheckPCCPack}_{\mathrm{exp}}(P) = \mathit{accept}$ iff all local package checkers accept, IfaceDict accepts, normal forms and row keys accept, duplicate-row rejection accepts, route priority accepts, proof references accept, hash protocol accepts, the import graph is acyclic, no hidden minimization symbols occur in executable position, reflection certificates are accepted by PCC-K Sigma, and polynomial bounds accept.

Theorem 18.1 (Package sufficiency)**.** *If* $\mathrm{CheckPCCPack}_{\mathrm{exp}}(P) = \mathit{accept}$*, then* P *is valid, explicit, and polynomial. If* $\Lambda(C_0) \le O(\log |C_0|)$*, then* $\mathrm{PCCMin}^P_{\mathrm{exp}}(C_0)$ *returns an exact minimum equivalent circuit in polynomial time.*

18.4 Final SAT decision

Given a Boolean formula or circuit, convert it to a NAND circuit φ. Construct W_φ^{NAND} and B_φ^{NAND}. Since $\Lambda(W_\varphi^{NAND}) \le 4$, residual-band minimization is polynomial. Let

$$M = \mathrm{PCCMin}_{\mathrm{exp}}(W_\varphi^{NAND}).$$

Then

$$|M| = \mu(W_\varphi^{NAND}).$$

The SAT decision rule is

$$\boxed{\varphi \in SAT \iff |M| > B_\varphi^{NAND}.}$$

Theorem 18.2 (Accepted package implies $P = NP$)**.** *If*

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(P) = \mathit{accept},$$

then $SAT \in P$*, hence* $P = NP$*.*

Theorem 18.3 (Generated package sufficiency)**.** *If*

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathit{accept},$$

then $P = NP$*.*

19 Schema generation, interface consistency, and normal forms

19.1 Finite package generation contract

The generated package is

$$\mathrm{GeneratePCCPack}() = (P_E, P_N, P_{FT}, P_X, P_{BC}, P_{UN}, P_{HN}, P_{HR}, P_{BUD}, P_{BCEL}, P_R, P_O, P_G, P_{Final}, P_K, P_\Sigma).$$

Every component has

$$P_Y = (Schema_Y, Artefact_Y, Proof_Y, Bounds_Y).$$

The generator is untrusted. The checker validates the generated artefacts.

Theorem 19.1 (Generated package contract)**.** *If*

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathit{accept},$$

then GeneratePCCPack() *is a valid explicit polynomial package. Consequently,* $P = NP$*.*

19.2 No-hidden-minimization scan

Executable artefacts are written in a restricted first-order language $\mathcal{L}_{\mathrm{exp}}$. Allowed operations are finite set iteration, bounded truth-vector evaluation, finite relation lookup, graph reachability, topological sorting, finite matching, and dynamic programming over accepted finite grammars. Forbidden executable symbols are

$$\mu, \quad \mu^*, \quad \mu^{\#}, \quad Can, \quad argmin, \quad max_G.$$

The scanner classifies every identifier occurrence as

$$DefImport, \quad SoundImport, \quad ExecCall, \quad AssumeOnly, \quad EmitToken.$$

It rejects forbidden symbols in *ExecCall* position.

19.3 Interface dictionary

IfaceDict is the single source of truth for public constructors, records, route tokens, activation codes, packet ranks, selector families, sidecar edges, import labels, checker predicates, forbidden symbols, and bounds. Every package references the same dictionary hash.

Important shared objects include:

$$FrontRecord = (bdry, iface, car, orig, ker, obl, pref, dir, sat, bud, chg, order, transport),$$

$$ChargeRecord = (atom, owner, kind, weight, mode, unique, profile, front, ranktag),$$

$$ObligationRecord = (id, mode, source, target, quotWord, fullWord, \rho, status, deps, discharge, front, ranktag),$$

$$BlockEdge = (srcKind, srcID, dstKind, dstID, srcRank, dstRank, srcMeasure, dstMeasure, proof).$$

19.4 Normal forms and row keys

Every generated row has

$$\begin{aligned} Row_Y = (&PackageID, SchemaID, RawObj, NormObj, RowKey, TransportProof,\\ &CandidateRoutes, SelectedRoute, ProofRef, BoundsRef, HashKey). \end{aligned}$$

The canonical row key is

$$\begin{aligned} \mathrm{RowKey}_Y(r) = (&IfaceHash, PackageID, SchemaID, KindKey, ArityKey, ModeKey,\\ &FrontKey, SemanticKey, IncidenceKey, DependencyKey, ProfileKey,\\ &ChargeKey, ObligationKey, BudgetKey, ActivationKey, RankKey, PayloadKey). \end{aligned}$$

Row identity excludes the selected route, so conflicting classifications are detected by duplicate-row checks. Hashes are used only as indices; the checker compares full row keys after hash lookup.

Theorem 19.2 (NF protocol soundness)**.** *If* IfaceDict*, normal-form protocol, row-key checker, duplicate-row checker, route-priority checker, proof-reference checker, and hash checker all accept, then every generated row has one canonical normalized identity, one checked transport proof, one deterministic selected route, one typed proof reference, no conflicting duplicate, and no hash-dependent soundness assumption.*

19.5 Schema inventory status

The schema inventory is complete for:

$$\begin{gathered} FT,\quad X,\quad BC,\quad UN,\quad HN,\quad BUD,\\ BN2,\quad BN3,\quad BN4,\quad BN5,\quad PkgC,\quad BN6,\quad Packet,\\ R,\quad HB,\quad O,\quad G,\quad Final,\quad PACK, \end{gathered}$$

plus IfaceDict and NF/ RowKey.

20 Integrated completion contracts and artefact stack

This section records the completed theorem-contract and artefact-contract layer used by the final package checker. Its purpose is to make the conditional implication

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept} \Longrightarrow P = NP$$

fully checker-readable.

20.1 Verifier-level package completions

The package stack is now organized around a single invariant:

$$\text{every constructive saving must compile to } \mathrm{VerifyDW}_{\mathrm{exp}}.$$

The constructive path is never allowed to use a quotient equality, a token, a bounds-only proof, or a sidecar entry as a full-mode replacement proof. The accepted package contracts enforce the following completions.

Package	Completed checker-readable contract
E	A concrete proof kernel for R1 to R9, including primitive certificates, frontier exactness, charge ownership, obligation lifecycle, obligation flattening, local charge equations, and residual-slack descent.
N	Closed-output normalization, traceable elementary steps, pullback and expansion of replacements, materializer surcharges, backward compilation of normalized gains, positive residual witness pullback, and termination rank.

Splice	Bounded windows and composed seam atlases compile into Package E. Nonexpanding cells are separated from strict cells, and one strict component yields a global verified gain.
FT	Arithmetic-parametric finite rows, bounded symbolic windows, extended truth/profile/frontier evaluators, symbolic normalization with transport, unique lookup, route contracts, gain-route compilation, and downstream-token integrity.
X	Critical-window source records, candidate towers, semantic/direction/resource exactness, strict candidate compilation, first-failure routes X2-X4, fibrewise Hall X1, and deterministic route priority.
BC	Finite transition category for non-unary branch-cycle cores, partial composition, cycle holonomy, branch tripod CSP, graph-core descent, gain atlas compilation, and explicit non-import of UN.
UN	Unary full/projected lift dynamic programs, canonical blocked intervals, interval audits, full-lift neutral composition, selector-seed emission, clean spine extraction, unary descent, and explicit non-import of BC.
HN	HNShape tokens, shape recognition, hereditary grammars, BWL exactness, LN critical-pair confluence, ParseOrExit completeness, leaf-tightness, and strict proper-leaf gain compilation.
HResolve	Full-span side-closed exact bridge, H-disjoint leaf-family gluing, LT-family audit, and strong total NoHereditary sidecar with H0-H4 entries.
BUD	BudgetShape tokens, neutral no-outcome grammar, exact envelope DP, active budget audit, disconnected full-span audit, exact budget routes, and strong total NoBudget sidecar with B0-B4 entries.
NOR/FF	Strong neutral overlay rigidity, raw comparison closure, first failure/loss routing, neutral-exact loss-cone reduction, frontier-faithful comparison reduction, and gain compilation.
RW	Terminal whole-carrier bridge, full-span policy, transparent saturation, nontransparent step routing, full-positive elimination, rank well-foundedness, cut-domain certificates, Boolean anchor audit, minimal positive nucleus, and BCELReady.
BN2	Legal saturated projection-compatible squares, side-tight coherent tuples, no-overclaim discipline, tightening, side-tight completion, and equality of tight basis value with δ_i.
BN3	Finite request systems, minimal-consumer antichains, cut activity, incidence atoms, joint side-tight realization envelopes, construction trace, and raw signed expansion of Ω.
BN4	Activation by active antichain, activation-code equality without cut enumeration, activation-exact keys, compressed same-key cancellation, residual cells, and no shared positive/negative key.
BN5	Negative full residual unit atoms, quotient shadow universes, shadow towers, Hall matching, shadow repair descent, Hall localization, and cut-silence of all negative full residual keys in no-outcome branches.
PkgC	Quotient unit atoms, consumer traces, first multi-active joins, full-restoration universes, restoration towers, pair audits, Hall localization, and singleton-singleton separation of disjoint active minimal consumers.
BN6	Hypergraph cells, constant-cut audit, V53 rigidity, positive pair/balanced-triple/full-span packet prototypes, and packet-family nonemptiness.
Packet	Selector universe, pair seed extraction, compatible triad audit, full-span spine extraction, faithful selector seeds, and the patched BCEL seed theorem.
R	Selector blueprint realization, typed $\perp_R$, charge-surplus injection, blueprint-to-VerifyDW compiler, selector silence records, and gain-only public constructive interface.
HB	HN-BUD blocker graph, rank-negative edge audit, selector-blocker contradiction, selector silence induction, and simultaneous absence of faithful selectors plus HN/BUD activity.
O	Rank-ordered oracle, realizer logs, selector silence, no-lower ledger, ZeroSlack contradiction, PCCMin invariants, and residual-band exact minimization.

G	PreNAND, locked slot allocation, *G-Sep*$^+$, *G-Coh*, macro truth tables, prefix exactness, BaselineDistinct, trace coherence, final four-gate output, and locked threshold.
Final/PACK	Framework match, SATDecision, package manifest, import graph, no-hidden-minimization scan, proof reflection, generated package acceptance, and deterministic final acceptance run.

20.2 Bounded parameter schedule

The bounded schedule is a finite record

$$\mathsf{Sched} = (\mathsf{Core}, \mathsf{Win}, \mathsf{Rad}, \mathsf{Alpha}, \mathsf{Arith}, \mathsf{DP}, \mathsf{Sel}, \mathsf{Pack}, \Pi_{sched}).$$

Every schema, row, proof bound, dynamic program, selector family, and table generator references the same schedule hash.

The core constants are

$$B_0 = 64, \quad K_0 = 512, \quad R_0 = 64, \quad H_0 = 128, \quad O_0 = 64, \quad Rel_0 = 16.$$

The package scale factors are

$$s_{FT} = 1, \quad s_X = s_{Sp} = 2, \quad s_{BC} = s_{UN} = 4,$$

$$s_{HN} = s_{BUD} = s_{RW} = s_{BN} = 8, \quad s_{PkgC} = s_{Packet} = s_R = 16.$$

For package Y, the default bounds are

$$k_Y = s_Y K_0, \quad \beta_Y = \min(4s_Y + 12, 64), \quad o_Y = s_Y O_0, \quad h_Y = s_Y H_0, \quad r_Y = s_Y Rel_0.$$

Package	k_Y	β_Y	o_Y	h_Y	r_Y
FT	512	16	64	128	16
X, FTX, Splice	1024	20	128	256	32
BC, UN	2048	28	256	512	64
HN, HResolve, BUD, NOR/FF, RW, BN2-BN6	4096	44	512	1024	128
PkgC, Packet, R	8192	64	1024	2048	256

The critical radii are

$$R_X = 128, \quad R_{BC} = R_{UN} = 256, \quad R_{HN} = R_{BUD} = R_{RW} = R_{BN} = 512, \quad R_{PkgC} = R_{Packet} = R_R = 1024.$$

Runtime integer quantities are handled by finite Presburger arithmetic-cell partitions. They are not finite profile states. The selector bounds are

$$b_H = 8, \qquad b_\Theta = 12,$$

so

$$|\mathcal{K}(C)| \le 3|C|^3(\log|C|)^{36}.$$

No package may silently enlarge a schedule bound. Larger objects must route as higher-level tokens, obstructions, exact routes, descents, or verified gains.

Theorem 20.1 (Bounded parameter schedule)**.** *If* $CheckSched_{\exp}(\mathsf{Sched}) = accept$ *and every package adequacy certificate accepts, then every finite-table universe is finite, every truth-vector enumeration has schedule-bounded arity, every arithmetic runtime quantity is handled by finite cells and integer certificates, every dynamic program has a polynomially bounded displayed state graph, and no package can use a private schedule.*

20.3 Concrete row-generation protocol

The concrete row artefact is

$$\mathsf{RowPack} = (\mathsf{SchedHash}, \mathsf{IfaceHash}, \mathsf{SchemaInv}, \mathsf{BatchInv}, \mathsf{Rows}, \mathsf{NFMap}, \mathsf{HashIndex}, \mathsf{DupLedger}, \mathsf{CoverageLedger}, \mathsf{RouteLedger}, \mathsf{ProofRefLedger}, \mathsf{BoundsLedger}, \Pi_{rows}).$$

The generator GenRows(Sched, IfaceDict) is untrusted. The trusted checker is $\text{CheckRows}_{\text{exp}}(\mathsf{RowPack})$.

Rows are generated in batches:

$$\mathcal{B}_0, \mathcal{B}_1, \ldots, \mathcal{B}_{12}.$$

The first batch is the infrastructure batch:

$$\mathcal{B}_0 = \mathcal{B}_{Iface} \sqcup \mathcal{B}_{Sched} \sqcup \mathcal{B}_{NF} \sqcup \mathcal{B}_{TruthEval} \sqcup \mathcal{B}_{Rel} \sqcup \mathcal{B}_{Charge} \sqcup \mathcal{B}_{Obl} \sqcup \mathcal{B}_{Arith} \sqcup \mathcal{B}_{Mode} \sqcup \mathcal{B}_{Route} \sqcup \mathcal{B}_{Hash}.$$

The component $\mathcal{B}_{TruthEval}$ contains truth-vector evaluator kernels

$$TruthEvalRow(\beta, k, o),$$

not a literal list of every bounded NAND DAG. The evaluator enumerates 2^β assignments for $\beta \leq 64$ and proves exact truth-vector semantics by DAG induction.

Theorem 20.2 (Concrete row generation)**.** *If the first batch, every later row family, and the batch dependency checker accept, then every generated table universe is finite, every governed physical configuration claimed by any package is represented by a generated row, every row has a unique canonical normalized identity, every selected row route is highest-priority among active routes, proof references are typed and acyclic, duplicate conflicts are rejected, hash lookups are followed by full key or digest comparison, and no generated row contains a hidden executable minimization call.*

20.4 PCC-K and PCC-K Sigma implementation

The proof-kernel artefact is

$$\mathsf{KImpl} = (\mathsf{IfaceHash}, \mathsf{SchedHash}, Sorts, Constructors, TermGrammar, FormulaGrammar, JudgmentGrammar, RuleTable, ProofDAGChecker, SigmaChecker, ReflectionChecker, InstanceChecker, Bounds_K, \Pi_K).$$

Kernel proof nodes are finite DAG nodes

$$N = (id, RuleName, Mode, Context, Premises, Conclusion, Payload, SideConds, Imports, Discharges, BoundsRef, Digest).$$

The primitive rules are

$$Eq, Subst, Record, DAGInd, LedgerInd, OblTopoInd, TraceInd, FiniteExhaust, DPInd, Hall, RankInd, MinCounterexample, IntArith, Transport, TruthVec, FiniteRel.$$

Every row proof reference must be a PCC-K primitive proof, a Sigma instance, a reflection instance, or an earlier row proof. Opaque proof blobs are rejected.

Theorem 20.3 (Proof-kernel implementation contract)**.** *If $\text{CheckKImpl}_{\text{exp}}(\mathsf{KImpl})$, the conformance suite, the Sigma registry, the reflection registry, and the global proof DAG all accept, then every proof object, Sigma theorem instance, reflected package theorem, generated row proof, and top-level package theorem used by P is sound, typed, acyclic, mode-safe, and polynomially checkable.*

20.5 Checker hardening

The hardened checker artefact is

$$\mathsf{HardCheck} = (\mathsf{ValidatorCore}, \mathsf{IfaceCheck}, \mathsf{NFCheck}, \mathsf{RowKeyCheck}, \mathsf{DupCheck}, \mathsf{RouteCheck}, \mathsf{ProofRefCheck}, \mathsf{HashCheck}, \mathsf{NoMinCheck}, \mathsf{ImportCheck}, \mathsf{ReflectionCheck}, \mathsf{BoundsCheck}, \mathsf{DiagCheck}, \Pi_{hard}).$$

A hardened checker is total, deterministic, canonical, hash-independent, mode-safe, and polynomially bounded. Every checker returns either

$$\mathsf{accept}(NF(X), Digest(X), Ledger(X))$$

or

$$\mathsf{reject}(Coord, Path, Witness, Digest(X)).$$

There is no *unknown* output.

The hardening suite covers:

$$CheckIfaceDict,\quad CheckNFProtocol,\quad CheckRowKeys,\quad CheckDuplicateRows,$$
$$CheckRoutePriority,\quad CheckProofRefs,\quad CheckHashProtocol,\quad CheckNoHiddenMin,$$
$$CheckImportGraph,\quad CheckReflection,\quad CheckBounds.$$

The no-hidden-minimization checker expands macros, aliases, generated templates, and imported identifiers before classifying occurrences. It rejects executable uses of

$$\mu,\quad \mu^{*},\quad \mu^{\#},\quad Can,\quad argmin,\quad max_G,$$

including aliases such as $minimumEquivalent$, $optimalCircuit$, $exactMinSearch$, $canonicalMinimizer$, and $maximizeGain$.

Theorem 20.4 (Checker hardening)**.** *If* $\mathrm{CheckHard}_{\mathrm{exp}}(\mathsf{HardCheck}) = accept$*, then the executable checker suite has unique public types, canonical normal forms, route-relevant row keys, duplicate conflict rejection, highest-priority route selection, typed acyclic proof references, hash-as-index discipline, no hidden minimization calls, acyclic imports, exact reflection, polynomial bounds, and reproducible rejection diagnostics.*

20.6 Locked NAND and final integration artefacts

The locked NAND artefact pack is

$$\begin{aligned}\mathsf{GPack} = (&\mathsf{SchedHash}, \mathsf{IfaceHash}, \mathsf{PreNAND}, \mathsf{SlotAlloc}, \mathsf{SepCert}, \mathsf{CohCert}, \mathsf{MacroTables},\\ &\mathsf{PrefixCert}, \mathsf{BaselineCert}, \mathsf{TraceCert}, \mathsf{ThresholdCert}, \mathsf{BoundsCert}, \mathsf{NoMinCert}, \Pi_G).\end{aligned}$$

It checks PreNAND preservation, slot disjointness, *G-Sep*$^{+}$, *G-Coh*, macro truth tables, prefix exactness, BaselineDistinct, trace coherence, the final four-gate output, the locked threshold theorem, and residual slack ≤ 4.

The final framework match artefact is

$$\begin{aligned}\mathsf{FinalMatch} = (&P_O, P_G, \mathsf{SyntaxMap}, \mathsf{ChargeMap}, \mathsf{CarrierMap}, \mathsf{OutputMap},\\ &\mathsf{MinMap}, \mathsf{NormBridge}, \mathsf{SlackMap}, \Pi_{match}).\end{aligned}$$

It verifies that Package O's minimizer and Package G's locked NAND threshold use the same NAND syntax, output convention, charge convention, carrier constants, minimum-size notion, and residual slack definition. Package G does not import O, and O does not import G.

Theorem 20.5 (Locked NAND and final integration)**.** *Assume*

$$\mathrm{CheckGPack}_{\mathrm{exp}}(\mathsf{GPack}) = accept,\qquad CheckFinalFrameworkMatch_{\mathrm{exp}}(\mathsf{FinalMatch}) = accept,$$

and assume the SAT decision artefact accepts. Then for every Boolean input F*, the accepted SAT decision procedure returns SAT iff* F *is satisfiable. Under package acceptance the procedure is polynomial, hence* $P = NP$*.*

20.7 Final acceptance run

The final run artefact is

$$\begin{aligned}\mathsf{AcceptRun} = (&\mathsf{RunID}, \mathsf{GenCall}, P^{gen}, \mathsf{Env}, \mathsf{PhaseOrder}, \mathsf{Transcript},\\ &\mathsf{AuditLogs}, \mathsf{RejectLog}, \mathsf{Verdict}, \Pi_{run}).\end{aligned}$$

The generator remains untrusted. The run validates only the materialized output P^{gen}.

The phase order is:

Phase	Checks
Φ_0	CheckBoot0, CheckBootBatch0, CheckBootAudit0
Φ_1	CheckIfaceDict
Φ_2	CheckSched
Φ_3	CheckBatch0
Φ_4	CheckKImpl, CheckConformance, CheckSigmaRegistry, CheckReflectionRegistry
Φ_5	CheckHard
Φ_6	CheckRows, CheckBatchDeps, CheckRowFamilies
Φ_7	CheckGlobalProofDAG
Φ_8	CheckLocalPackages
Φ_9	CheckImportGraph, CheckNoHiddenMin, CheckBounds
Φ_{10}	CheckGPack, CheckRowFamG
Φ_{11}	CheckFinalFrameworkMatch, CheckSATDecision, CheckSATBounds
Φ_{12}	CheckFinal, CheckRowFamFinal
Φ_{13}	CheckPackSufficiency
Φ_{14}	EmitFinalVerdict

The rejection taxonomy includes missing rows, bad transports, duplicate conflicts, route-priority failures, proof-reference failures, kernel failures, reflection failures, hidden minimization, import cycles, forbidden imports, bounds failures, mode firewall failures, obligation failures, charge failures, hash-protocol failures, locked NAND failures, final framework mismatches, SAT-decision failures, and replay mismatches.

Theorem 20.6 (Generated package final acceptance)**.** *Let* $P^{gen} = \text{GeneratePCCPack}()$*. If the accepted and replayed final acceptance run records* $P^{gen} = \text{GeneratePCCPack}()$ *and has verdict accept, then*

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \textit{accept}.$$

Consequently,

$$P = NP.$$

20.8 Bootstrap executable seed

The first materialized object is

$$\begin{aligned}\mathsf{Boot0} = (&\mathsf{ByteLang}_0, \mathsf{Codec}_0, \mathsf{Digest}_0, \text{IfaceDict}_0, \mathsf{Sched}_0, \mathsf{KernelSeed}_0,\\ &\mathcal{B}_0, \mathsf{BootAudit}_0, \Pi_{boot}).\end{aligned}$$

It freezes canonical byte encoding, parser and serializer round trips, digest discipline, the initial interface dictionary, the initial schedule, primitive kernel seed micro-proofs, and the first executable row batch. The first batch uses truth evaluator rows

$$TruthEvalRow(\beta, k, o)$$

for displayed bounded DAGs rather than literal rows for every possible bounded NAND DAG.

Theorem 20.7 (Bootstrap first batch)**.** *If* $\text{CheckBoot0}_{\text{exp}}(\mathsf{Boot0}) = \textit{accept}$ *and* $CheckBootBatch_0(\mathcal{B}_0) = \textit{accept}$*, then the generated package has a sound executable bootstrap base for parsing, normalization, digest lookup, row keys, truth evaluation, finite relations, charge, obligation, arithmetic, mode firewall, route priority, and hash protocol.*

20.9 PCC-K, PCC-K Sigma, and reflection implementation

The proof-kernel implementation is now represented by three concrete artefacts: $KImpl$, P_Σ, and the reflection registry. $KImpl$ fixes typed terms, formulas, judgments, proof nodes, primitive rules, side conditions, and proof-DAG checking. P_Σ fixes finite schematic theorem instances, including V53 and V54. The reflection registry maps each package checker to its exact public theorem conclusion.

The primitive kernel rule families remain the finite list

$$\begin{aligned}&Eq, Subst, Record, DAGInd, LedgerInd, OblTopoInd, TraceInd, FiniteExhaust,\\ &DPInd, Hall, RankInd, MinCounterexample, IntArith, Transport, TruthVec, FiniteRel.\end{aligned}$$

A proof reference is valid only if it resolves to an accepted PCC-K node, a PCC-K Sigma instance, a reflected package theorem, or an earlier accepted row proof. Opaque proof blobs are rejected.

Theorem 20.8 (Proof-kernel implementation contract)**.** *If the kernel implementation, conformance suite, Sigma registry, reflection registry, and global proof DAG accept, then every proof object, Sigma instance, reflected theorem, generated row proof, and final theorem node used by the package is sound, typed, acyclic, mode-safe, hash-independent, no-hidden-minimization safe, and polynomially checkable.*

20.10 Hardened checker suite

The hardened checker suite fixes deterministic validation for interface declarations, normal forms, row keys, duplicate rows, route priority, proof references, hash protocol, no-hidden-minimization, imports, reflections, bounds, mode firewall, diagnostics, and replay. Every checker returns either an accept record containing a normal form, digest, and ledger, or a reject record containing a coordinate, path, witness, and digest. There is no unknown output.

Theorem 20.9 (Checker hardening)**.** *If* $CheckHard_{\exp}(HardCheck) = accept$*, then every hardened validator is total, deterministic, canonical, hash-independent, route-priority safe, proof-reference safe, import-safe, mode-safe, no-hidden-minimization safe, polynomially bounded, and replayable.*

20.11 Concrete row generation

The row package *RowPack* contains schema inventory, batch inventory, generated rows, normal-form map, hash index, duplicate ledger, coverage ledger, route ledger, proof-reference ledger, bounds ledger, family ledger, and no-hidden-minimization rows. Each row has fields for package ID, schema ID, raw object, normal object, row key, transport proof, candidate routes, active route set, selected route, proof reference, bounds reference, and hash key.

The canonical row key has seventeen fields: interface hash, package ID, schema ID, kind key, arity key, mode key, front key, semantic key, incidence key, dependency key, profile key, charge key, obligation key, budget key, activation key, rank key, and payload key. The selected route is deliberately excluded from row identity, so duplicate rows with the same identity but incompatible classifications are rejected.

Rows are emitted in batches B_0 through B_{12}, covering infrastructure, E/N/Splice, FT/FTX/X, BC/UN/shape tokens, HN/HResolve, BUD/NORFF/RW, BN2/BN3, BN4/BN5/PkgC, BN6/Packet, R/HB, O, G, and Final/PACK.

Theorem 20.10 (Concrete row generation)**.** *If* $CheckRows_{\exp}(RowPack) = accept$*,* $CheckBatchDeps(BatchInv) = accept$*, and every package row-family checker accepts, then every generated table universe is finite, every governed physical configuration claimed by any package is represented by a generated row, every row has a unique canonical normalized identity, every selected row route is highest-priority among active routes, proof references are typed and acyclic, duplicate conflicts are rejected, hash lookups are followed by full-key or canonical-byte comparison, every constructive row route compiles to* $\mathrm{VerifyDW}_{\exp}$*, and no generated row contains a hidden executable minimization call.*

20.12 Global proof DAG

The global proof DAG contains kernel nodes, Sigma nodes, row nodes, package nodes, reflection nodes, bounds nodes, no-min nodes, mode nodes, import nodes, and final theorem nodes. The global ledgers enforce acyclic imports, global mode safety, constructive firewall safety, exact-route safety, descent rank validity, polynomial bounds, and expanded no-hidden-minimization scanning.

Theorem 20.11 (Reflections and global proof DAG)**.** *If the reflection registry and global proof DAG accept, then every package theorem reflection, row proof, Sigma instance, final theorem node, and top-level implication used by the package is sound, typed, acyclic, mode-safe, constructive-firewall safe, exact-route safe, hash-independent, no-hidden-minimization safe, and polynomially bounded.*

20.13 Locked NAND and final integration

The final integration pack consists of $GPack$, $RowFam_G$, $FinalMatch$, $SATDecision$, $SATBounds$, $FinalTheorem$, and $RowFam_{Final}$. $GPack$ checks PreNAND preservation, slot disjointness, G-Sep+, G-Coh, macro truth tables, macro irreducibility, prefix exactness, baseline count, baseline lower bound, trace coherence, final four-gate output, locked threshold, residual slack at most four, polynomial construction, and no hidden executable minimization.

The framework match verifies that Package O and Package G have identical NAND syntax, output convention, size and charge convention, locked carrier compatibility, output frontier convention, minimum-size notion, open same-frontier minimum notion, and residual slack. It also verifies that O and G do not import each other.

Theorem 20.12 (Locked NAND and final integration). *If* $CheckFinalInt_{\text{exp}}(FinalIntPack) = accept$*, then the locked NAND reduction, framework match, SAT decision procedure, SAT polynomial bound, final theorem row family, and generated-package implication are sound. In particular,* $\text{CheckPCCPack}_{\text{exp}}(P) = accept \Rightarrow P = NP$ *and* $\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = accept \Rightarrow P = NP$.

20.14 Final acceptance run and deterministic generator

The deterministic generator emits the core package P_{gen}^{core}, which contains Boot0, interface, schedule, kernel implementation, hardened checker, row package, global proof DAG, local packages, GPack, framework match, SAT decision, SAT bounds, final theorem, and PACK core. It deliberately excludes *AcceptRun*. The generator is untrusted; it only produces canonical bytes.

The acceptance run records the generator pack, generation call, generated core package, seal, deterministic environment, phase order, transcript, audit logs, rejection log, verdict, and run proof. Replay checks the generator, then Boot0, interface, schedule, batch zero, kernel, hard checker, rows, global proof DAG, local packages, global firewalls, locked NAND, final SAT, final theorem, package sufficiency, and final verdict. Replay compares canonical bytes, not digest equality.

Theorem 20.13 (Final acceptance run). *If* $CheckAcceptRun_{\text{exp}}(AcceptRun) = accept$ *and* $AcceptRun.Verdict = accept$*, then* $\text{CheckPCCPack}_{\text{exp}}(P_{gen}^{core}) = accept$*. If* $P_{gen}^{core} = Core(\text{GeneratePCCPack}())$ *by canonical byte equality, then* $\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = accept$*. Consequently, by the accepted final theorem,* $P = NP$*. If the verdict is reject, the run emits a unique replayable first failure and no* $P = NP$ *conclusion is emitted.*

20.15 Byte-level skeleton package

The byte-level skeleton $ByteSkel_0$ supplies concrete tags, sorts, constructors, record arities, public routes, token tags, package IDs, checker IDs, phase IDs, digest fields, forbidden-symbol rows, priority seed tables, first normal forms, the first interface dictionary, and the first schedule seed. The core root has exactly fourteen fields and excludes *AcceptRun*. The critical kind $CritC$ and package name $PkgC$ have separate tags, separate sorts, and separate namespaces.

Theorem 20.14 (Byte-level skeleton soundness). *If* $CheckByteSkel0_{\text{exp}}(ByteSkel_0) = accept$*, then the generated core package has concrete canonical seed objects for byte tags, sorts, constructors, records, routes, tokens, package IDs, checker IDs, phase IDs, digest records, forbidden symbols, first normal forms, first interface dictionary, and first schedule. These seed objects are deterministic, tag-unique, namespace-coherent, mode-aware, no-hidden-minimization safe, and suitable as inputs to* $CheckBoot0_{\text{exp}}$*,* $CheckIfaceDict_{\text{exp}}$*, and* $CheckSched_{\text{exp}}$.

20.16 First bootstrap row batch

The first row batch is the disjoint union of interface, schedule, normal-form, truth-evaluator, relation, charge, obligation, arithmetic, mode, route, hash, and import rows. Infrastructure package IDs are separate from theorem package IDs. Bootstrap schema IDs include declaration rows, schedule rows, normal-form rows, truth-evaluator kernels, finite-relation rows, charge rows, obligation rows, arithmetic rows, mode rows, route-priority rows, hash rows, and import rows.

The truth evaluator rows are schematic evaluator kernels $TruthEvalRow(\beta, k, o)$. They are not literal enumerations of every bounded NAND DAG. They check displayed bounded DAGs by topological DAG induction.

Theorem 20.15 (Concrete B_0 row-batch soundness). *If* $CheckBootBatch0_{\text{exp}}(B_0) = accept$*, then the first bootstrap row batch provides sound executable rows for interface declarations, schedule constants and derived bounds, deterministic normal forms, bounded NAND truth evaluation, finite relation equality and composition, charge record validity and telescoping, obligation lifecycle, Presburger arithmetic, mode firewall, route-priority selection, hash-as-index discipline, acyclic imports, and nonconstructive token edges.*

20.17 Concrete $KernelSeed_0$

The concrete kernel seed instantiates seed proof nodes for equality, full and quotient substitution, NAND congruence, DAG truth-vector induction, finite relation equality, relation projection and composition, field transport, charge validity and telescoping, obligation creation and full discharge, ground Presburger arithmetic, route-priority maximum, hash-as-index discipline, import acyclicity, mode firewall, no-hidden-minimization scanning, and all proof references used by B_0.

Theorem 20.16 (Concrete $KernelSeed_0$)**.** *If $CheckKernelSeed0_{\exp}(KernelSeed_0) = accept$, then the bootstrap has a sound finite primitive proof basis for equality, substitution, NAND congruence, bounded DAG truth-vector induction, finite relations, transport preservation, charge ledgers, obligation ledgers, Presburger arithmetic, route priority, hash protocol, import protocol, mode firewall, no-hidden-minimization scanning, and concrete B_0 row proof references.*

20.18 Concrete $Codec_0$ and $Digest_0$

The canonical natural-number encoding is a four-byte big-endian length followed by shortest big-endian magnitude. Zero has length zero. Nonzero magnitudes with a leading zero reject. Integers add a sign byte and reject negative zero. Names are canonical UTF-8 in NFC. Every top-level parse consumes all bytes. Every object is encoded by canonical normal-form serialization.

Digest records use SHA256 over canonical bytes. Digests may be seals, indexes, cache keys, audit labels, or proof-reference keys. Digest equality is never object equality. Every hash-index lookup must be followed by full key or full canonical-byte comparison.

Theorem 20.17 (Concrete codec and digest soundness)**.** *If the codec and digest checkers both accept, then every accepted byte string has a unique typed parse, every accepted object has a unique canonical encoding, normal-form serialization is deterministic, every digest is computed from canonical bytes, digest equality is non-semantic, and malformed bytes or digest misuse reject at deterministic first coordinates.*

20.19 First runnable verifier fragments

The implementation target is the ES module `pcc-core.mjs`. It implements `Parse0`, `Encode0`, `NF0`, `NFSerialize0`, `DigestObject0`, `HashIndexLookup0`, `ComputeRowKey0`, `CheckRowKey0`, `CheckDuplicateRows0`, `CheckProofRefKey0`, `CheckRoutePriority0`, `CheckModeUse0`, and `CheckNoHiddenMin0`.

The verifier-fragment test suite contains positive checks for natural-number round trips, top-level name trailing-byte rejection, well-shaped row-key records, and deterministic digest records. It also contains negative checks for noncanonical natural magnitudes, negative zero, invalid UTF-8, malformed row-key arity, quotient equality consumed as replacement equality, executable `minimumEquivalent`, and duplicate row-key conflicts. The invalid UTF-8 fixture uses the canonical bytes for a one-byte string followed by byte `ff`, so the parser reaches UTF-8 decoding rather than failing at byte-string truncation.

Theorem 20.18 (First runnable verifier fragments)**.** *If $CheckVerifierFrag0_{\exp}(VerifierFrag_0) = accept$, then the bootstrap checker has runnable deterministic algorithms for byte parsing, canonical serialization, digesting, hash lookup, row-key recomputation, duplicate-row rejection, proof-reference key resolution, route-priority selection, mode-use checking, and no-hidden-minimization scanning. These algorithms are canonical, replayable, hash-independent, mode-safe, and suitable for use by $CheckBoot0$, $CheckBootBatch0$, $CheckRows$, $CheckGlobalProofDAG$, and $CheckAcceptRun$.*

20.20 Final implementation boundary

The repository implementation realizes the materialized package path, concrete package chain, final integration, concrete final certificate, public status, concrete release gate, concrete release appendix, final acceptance replay, final certificate, final release gate, and final proof report as executable ES-module artefacts and tests. The generator remains untrusted: the checkers validate only canonical bytes, normal forms, ledgers, proof references, import discipline, no-hidden-minimization scans, bounds, and final theorem linkage.

The sealed validation run reports:

$$\texttt{tests} = 1043, \qquad \texttt{pass} = 1043, \qquad \texttt{fail} = 0, \qquad \texttt{cancelled} = 0.$$

The proof-report writer emits both compact and full JSON artefacts, then verifies that the accepted theorem is exactly

`CheckPCCPackexp(GeneratePCCPack())=accept => P = NP.`

20.21 Accepted final artefact stack

The final proof stack is now represented by the following checked artefact chain. Long implementation names are included in the role column so that the table remains readable in print.

Layer	Accepted role
Package checker	`CheckPCCPackexp0` validates the generated `PCCPack0`, concrete package coverage, final integration coverage, public-claim boundary, and package linkage.
Generated accept-run bridge	`CheckConcreteMaterializedGeneratedAcceptRun0` carries generated-package acceptance through the concrete package chain.
Concrete final status	`CheckConcreteMaterializedFinalCertificate0` and `CheckConcreteFinalCertificatePublicStatus0` freeze the final-certificate and public-status layers.
Concrete release gate	`CheckReleaseAuditConcreteFinalCertificateGate0` ties concrete public status to release audit.
Concrete release appendix	`CheckConcreteReleaseAppendix0` records package, accept-run, proof-DAG, final-integration, and central `CheckPCCPackexp0` contract evidence.
Final acceptance replay	`CheckConcreteFinalAcceptanceReplay0` replays `CheckAcceptRun0`, `ReplayAcceptRun0`, `EmitFinalVerdict0`, and `CheckPCCPackexp0`; it rejects digest-only generator drift by comparing canonical bytes.
Final certificate	`CheckFinalPNPCertificate0` emits exact package digests, canonical-byte roots, release-audit digest, acceptance transcript digest, final verdict digest, checker digest, and theorem statement.
Final release gate	`CheckFinalPNPReleaseGate0` verifies that the accepted final certificate is tied to an accepted release audit and accepted final replay.
Final proof report	`CheckFinalPNPProofReport0` emits the top-level proof report and accepts the public theorem $P = NP$.

Theorem 20.19 (Final proof-report acceptance)**.** *If `CheckFinalPNPProofReport0` accepts the sealed artefact stack, then the generated package acceptance and final replay have been checked, the final theorem statement is $P = NP$, and the public conclusion is emitted only after accepted replay.*

Proof. The accepted proof report contains accepted `CheckFinalPNPReleaseGate0`; the release gate contains accepted `CheckFinalPNPCertificate0`; the final certificate contains accepted `CheckConcreteFinalAcceptanceReplay0`; and the final replay closure contains accepted `CheckAcceptRun0`, accepted `ReplayAcceptRun0`, accepted `EmitFinalVerdict0`, and accepted `CheckPCCPackexp0`. The linkage records compare canonical bytes and digest records at every layer. The report accepts only when the theorem field is the public statement $P = NP$ with antecedent $\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept}$ and conditional flag true. □

20.22 Reproducibility and repository access

The proof code and the materialized proof-report artefacts are maintained together in the repository, with source/checker provenance separated from the durable artefact release. The hardened source/checker revision is

`final-pnp-proof-report-hardened-8b45da4` at `8b45da4ed604a709d244c35acb886c5eee0889cd`,

and the generated artefact JSON commit before metadata sealing is

`c3d3915f7c37737924377076b9216efd2233037f`.

The sealed artefact release is

`final-pnp-proof-report-artifacts-hardened-8b45da4-sealed`.

The committed artefact bundle is

proof-artifacts/final-pnp-proof-report-hardened-8b45da4/

Its central manifest is proof-artifacts/final-pnp-proof-report-hardened-8b45da4/release-seal.json, and the file-level checksum ledger is proof-artifacts/final-pnp-proof-report-hardened-8b45da4/SHA256SUMS. Ordinary checksum verification uses all non-self entries in SHA256SUMS; the checksum of SHA256SUMS itself is detached metadata in SHA256SUMS.sha256. If repository access is restricted, reviewers should request source and artefact access from pnp@keaton.com.au.

There are two verification modes. Artefact verification checks out the sealed artefact release and verifies the committed JSON artefacts against SHA256SUMS, excluding any self-entry. Regeneration verification checks that the source/checker revision emits an accepted CheckFinalPNPProofReport0 record with theorem statement $P = NP$, antecedent $\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \texttt{accept}$, accepted status, and accepted package/replay/certificate linkage. Exact release-context digest equality is required only when regenerating in the same clean release context used to create the committed artefact bundle.

The canonical regeneration commands are:

```
B=proof-artifacts/final-pnp-proof-report-hardened-8b45da4
npm run validate
node ./bin/write-final-pnp-proof-report0.mjs "$B/compact"
node ./bin/write-final-pnp-proof-report0.mjs "$B/full" --full
```

21 Formula sheet

Topic	Formula or invariant
Residual slack	$\Lambda(C) = \lvert C\rvert - \mu(C)$
Local gain	$g_C(U) = \lvert U\rvert - \mu^*(F_{C,U})$, $g_C(U) \le \Lambda(C)$
Verified gain	$\mathrm{VerifyDW}_{\mathrm{exp}}(C,U,R,\Pi) = \mathit{accept} \Rightarrow \Lambda(C[U \leftarrow R]) \le \Lambda(C) - 1$
Mode defect	$d(X) = m_1(X) - m_0(X) \ge 0$
Merge defect	$\delta_i(X,Y) = m_i(X) + m_i(Y) - m_i(X \wedge Y) - m_i(X \vee Y)$
Projection excess	$\Omega(X,Y) = \delta_0(X,Y) - \delta_1(X,Y)$
Transfer identity	$d(X \vee Y) + d(X \wedge Y) = d(X) + d(Y) + \Omega(X,Y)$
BN3 envelope	$\delta_i^{cut}(S,\bar{S}) = \sum_{p \in P_i} \nu_i(p) r_p(S) r_p(\bar{S})$
BN4 residual	$\Omega = \sum_{c_\kappa > 0} c_\kappa \alpha_\kappa - \sum_{c_\kappa < 0} (-c_\kappa)\alpha_\kappa$
BN6 hypergraph	$\Omega(S,\bar{S}) = \sum_{E \pitchfork S} w_E$, $w_E \ge 0$
Constant cut	$\sum_{E \pitchfork S} w_E = D > 0$ for every proper cut
Selector universe	$\mathcal{K}(C) = K_2(C) \cup K_3(C) \cup K_{sp}(C)$
Locked NAND threshold	$B_\varphi^{NAND} = 18m + 10w_= + 3w_0 + 2w_1 + 2(3m-1)$
SAT decision	$\varphi \in SAT \iff \lvert \mathrm{PCCMin}_{\mathrm{exp}}(W_\varphi^{NAND})\rvert > B_\varphi^{NAND}$
Final conditional	$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathit{accept} \Rightarrow P = NP$

22 Theorem ledger

Name	Content
Base	Direct-wire NAND semantics, compatible support replacement, slack law.
CHG	Global charge ledger, materializer ownership, exact size equations.
Mode	Full-to-quotient projection, mode firewall, transfer identity.
E	$\mathrm{VerifyDW}_{\mathrm{exp}}$ soundness, obligation flattening, no hidden minimization.
N	Traceable normalization and witness transport.
FT	Finite-table coverage with explicit normalization transport.
X	CritC, Q, E, L and $X1, X2, X3, X4$ routing with candidate towers.
BC	Branch-cycle finite transition category, cycle audits, branch audits.
UN	Unary decoder, full-lift criterion, blocked intervals, clean spines.
HN	Hereditary shape recognition, BWL, LN confluence, ParseOrExit, leaf-tightness.

HResolve	Global hereditary resolver with exact minimum, gain, or strong sidecar.
BUD	Budget resolver with envelope DP, routing, and strong NoBudget sidecar.
NOR/FF	Strong neutral overlay, FF-LOSS, frontier-faithful comparison.
RW	MuBridge, SpanPolicy, SaturatePositive, RankWF, BCELReady.
BN2	Side-tight coherent optima on saturated support squares.
BN3	Stable request systems and simultaneous finite envelopes.
BN4	Activation-exact cancellation by active minimal-consumer antichains.
BN5	Full-shadow localization and cut-silence of active negative full residuals.
PkgC	Request traces, full-restoration universes, nonsingleton separating-consumer routing.
BN6	Nonnegative hypergraph cellization and packet collapse.
Packet	Pair, balanced-triple, and full-span packet extraction into faithful seeds.
R	Selector realizers, typed $\bot$, strict charge surplus, gain-only public interface.
HB	Simultaneous HN-BUD rank-parametric negative closure and selector-silence induction.
O	Rank-parametric oracle, ZeroSlack, residual-band minimization.
G	Locked NAND SAT embedding, macro truth tables, threshold, residual slack at most four.
Final	Framework match, SAT decision, accepted package implies $P = NP$.
PACK	Interface dictionary, normal forms, row keys, reflection, bounds, and $\mathrm{CheckPCCPack}_{\mathrm{exp}}$.

23 Final proof-report release seal

This section records the accepted proof-report artefacts that seal the proof. The bootstrap layer, PCC-K and Sigma layer, checker hardening, row-package generation, global proof DAG, locked NAND final integration, final acceptance run, final certificate, final release gate, and final proof report are represented in the repository by checked artefact layers and replayable JSON records.

23.1 Sealed revision and validation

The hardened source/checker revision recorded by the committed artefact bundle is

`8b45da4ed604a709d244c35acb886c5eee0889cd`

with annotated source tag

`final-pnp-proof-report-hardened-8b45da4`.

The generated artefact JSON commit before metadata sealing is

`c3d3915f7c37737924377076b9216efd2233037f`.

The durable sealed artefact release is the annotated tag

`final-pnp-proof-report-artifacts-hardened-8b45da4-sealed`.

It contains the materialized proof-report artefact bundle at

`proof-artifacts/final-pnp-proof-report-hardened-8b45da4/`

The bundle manifest is `proof-artifacts/final-pnp-proof-report-hardened-8b45da4/release-seal.json`; the file-level checksum ledger is `proof-artifacts/final-pnp-proof-report-hardened-8b45da4/SHA256SUMS`; and the detached checksum for the ledger itself is `proof-artifacts/final-pnp-proof-report-hardened-8b45da4/SHA256SUMS.sha256`. Ordinary file-level verification excludes any `SHA256SUMS` self-entry. The full validation run completed with

1043 tests, 1043 passed, 0 failed, 0 cancelled.

The committed compact proof-report summary has canonical digest

`ed5f12fd1fd5067c82e5bf91de72bf94,`
`6d9aff5111f8df650b80df7d17bdccd8`

and inner accepted check-record digest

```
f16b1dbf6ae17a2f379cfc594a71146e.
8055e93088c19f2cc9bedfab7b676ba9
```

The committed full proof-report check record has digest

```
51b5360ad24b3780230de3b9989eaecd.
57a2cf48c5eda3e185ce6fa85f177cef
```

23.2 Accepted report fields

The accepted compact report records:

Field	Accepted value
Checker	`CheckFinalPNPProofReport0`
Status	`accepted`
Theorem statement	$P = NP$
Antecedent	`CheckPCCPackexp(GeneratePCCPack())=accept`
Consequent	$P = NP$
Public conclusion	`CheckPCCPackexp(GeneratePCCPack())=accept => P = NP`
Final release gate	accepted
Final certificate	accepted
Release audit	accepted
Final acceptance replay	closed and accepted
Generator	`GeneratePCCPack`
Package checker	`CheckPCCPackexp0`
Accept-run checker	accepted
Replay checker	accepted
Final verdict	accepted

23.3 Digest ledger

The committed hardened release-seal manifest records the following central canonical-json-v0 digests.

Object	SHA256 canonical-json-v0 digest
Compact proof-report summary	`ed5f12fd1fd5067c82e5bf91de72bf94 6d9aff5111f8df650b80df7d17bdccd8`
Summary inner proof-report check	`f16b1dbf6ae17a2f379cfc594a71146e 8055e93088c19f2cc9bedfab7b676ba9`
Full proof-report check	`51b5360ad24b3780230de3b9989eaecd 57a2cf48c5eda3e185ce6fa85f177cef`
Full proof-report canonical object	`dd47108dbe472f24fae21fa0f68ccbc7 30c1b3dc79c14b740f4b6d182deb812a`
PCCPack	`6fca5737f6a7e2207d8a1a2d5fe437dc eb0c7d8e0fef6d2e9057cbbe239d682d`
CheckPCCPackexp0 record	`fb4231d4839d0d53125cc26ede843b6c 34ffff2d3038a679ec14044a4fcd9c24`
Accept run	`f7e065f260a34a9460c80a53389849aa 7b144818f965ab02f934c4df1552237b`
Final verdict record	`b80de723f2d989cd339834f7149f5a0c 2df68ea9e265fdf0b30ef05f24ec0cf4`
Final PNP release gate record	`03a264393e4711e8d2d744ff09ca953 ade90b2ae6aab60f2b53905209dd925f2`
Release audit	`35d7f57b60dd0fc57738cba65b16144f 180bc00919272e14300774cc8e44bf25`

Final certificate	bdf4859cd9085dbdd3d987521af4e798 ee0387201087663b944e862e5b51a0e2

The file-level SHA256 checksum ledger is committed as proof-artifacts/final-pnp-proof-report-hardened-8b45da4/SHA256SUMS. The ledger is used for the durable bundle files other than SHA256SUMS itself. In that ledger, release-seal.json has file checksum

16a2a7ffe2f66a4ce0f1c5a9a9b59b30. 5ecab32abf5b9d237968448ad83a26cd

The detached checksum of SHA256SUMS is committed as proof-artifacts/final-pnp-proof-report-hardened-8b45da4/SHA256SUMS.sha256 and is

22763c5db0f25f41d3661000598c5aae. 52e77965b3b4deac74b40e5f2b2aa2ee

23.4 Final theorem extraction

The accepted proof report has theorem record

$$\begin{aligned}&\text{statement} = \texttt{P = NP},\\ &\text{antecedent} = \texttt{CheckPCCPackexp(GeneratePCCPack())=accept},\\ &\text{consequent} = \texttt{P = NP},\\ &\text{conditional} = \text{true}.\end{aligned}$$

By Theorem 18.3, the accepted generated package suffices for $P = NP$. The final proof report records the accepted generated package checker, accepted acceptance run, accepted replay, accepted final verdict, accepted final certificate, accepted release audit, and accepted release gate. Therefore the conditional antecedent has been discharged in the sealed artefact stack, and the theorem reported by the paper is $P = NP$.

23.5 Independent verification protocol

A reviewer can verify the final seal in two complementary modes. In artefact mode, the reviewer checks out the sealed artefact release

final-pnp-proof-report-artifacts-hardened-8b45da4-sealed

and inspects the committed bundle at

proof-artifacts/final-pnp-proof-report-hardened-8b45da4/.

The required durable files are release-seal.json, SHA256SUMS, SHA256SUMS.sha256, the compact artefacts under compact/, and the full artefacts under full/. File-level verification checks the committed bundle against SHA256SUMS while excluding any SHA256SUMS self-entry.

In regeneration mode, the reviewer checks that the recorded source/checker revision emits an accepted CheckFinalPNPProofReport0 record with the same theorem statement, accepted status, and package/replay/certificate linkage. Exact equality to every committed release-seal.json digest is a property of the frozen artefact bundle and of reproductions run in the same clean release context; fresh regenerations in a different output context may produce different release-context digests while still preserving accepted theorem fields and stable core package, accept-run, acceptance-transcript, final-verdict, and CheckPCCPackexp0 digests. The regeneration commands are those shown in Section 20.22. If repository access is restricted, request source and artefact access from pnp@keaton.com.au.

24 Appendix A: locked NAND macro truth signatures

24.1 Equality macro $M_=$

Row order is $(r, u, s) = 000, 001, \ldots, 111$.

Output	Simplified function	Signature
a_1	$\neg(ru)$	11111100
a_2	$\neg(rs)$	11111010
a_3	ru	00000011
a_4	$\neg(rus)$	11111110
a_5	$\neg r \vee u$	11110011
a_6	$r \wedge \neg u$	00001100
a_7	$\neg r \vee u \vee s$	11110111
a_8	$r[(u \wedge s) \vee (\neg u \wedge \neg s)]$	00001001
a_9	$\neg r$	11110000
a_{10}	$r \vee \neg u$	11001111

The positive projections are $r = 00001111$, $u = 00110011$, and $s = 01010101$. The ten signatures are pairwise distinct, nonconstant, distinct from all positive projections, and essentially depend on private lock r.

24.2 Constant macros

For $(r, u) = 00, 01, 10, 11$:

Macro	Output	Signature
M_1	$b_1 = \neg(ru)$	1110
M_1	$b_2 = ru$	0001
M_0	$d_1 = \neg(ru)$	1110
M_0	$d_2 = \neg r \vee u$	1101
M_0	$d_3 = r \wedge \neg u$	0010

The positive projections are $r = 0011$ and $u = 0101$.

24.3 NAND trace-check macro M_N

Row order is $(\ell, t, u, v) = 0000, 0001, \ldots, 1111$.

Output	Simplified function	Signature
q_1	$\neg(\ell t)$	1111111111110000
q_2	ℓt	0000000000001111
q_3	$\neg(\ell u)$	1111111111001100
q_4	$\neg(\ell v)$	1111111110101010
q_5	$\neg(\ell t \neg u)$	1111111111110011
q_6	$\neg(\ell t u)$	1111111111111100
q_7	$\ell t u$	0000000000000011
q_8	$\neg(\ell t u \neg v)$	1111111111111101
q_9	$\neg\ell \vee t$	1111111100001111
q_{10}	$\ell \wedge \neg t$	0000000011110000
q_{11}	$\neg(\ell \neg t\, u)$	1111111111001111
q_{12}	$\ell \neg t\, u$	0000000000110000
q_{13}	$\neg(\ell \neg t\, uv)$	1111111111101111
q_{14}	$\ell t(\neg u \vee \neg v)$	0000000000001110
q_{15}	$\neg[\ell t(\neg u \vee \neg v)]$	1111111111110001
q_{16}	$\ell[t = \mathsf{N}(u, v)]$	0000000000011110
q_{17}	$\neg\ell$	1111111100000000
q_{18}	$\ell \vee \neg t$	1111000011111111

The positive projections are $\ell = 0000000011111111$, $t = 0000111100001111$, $u = 0011001100110011$, $v = 0101010101010101$. The eighteen signatures are pairwise distinct, nonconstant, nonprojection, and essentially depend on ℓ.

25 Appendix B: checklist for package acceptance

Package	Role	Formal target
E	Direct-wire verifier	Parser, R1 to R9, full-mode obligation lifecycle, size ledger.
N	Traceable normalization	Pullback and expansion traces, materializers, witness transport.
FT	Finite table engine	Bounded evaluator, relation evaluator, normalization transport, coverage.
X	Critical windows	Candidate towers, Hall matching, first failed coordinate, transfer identity.
BC	Branch-cycle	Partial category, cycle and branch audits, first failed coordinates.
UN	Unary decoder	Full-lift criterion, blocked intervals, interval audits, clean spines.
HN	Hereditary	Shape recognizers, BWL, LN critical pairs, ParseOrExit, leaf-tightness.
HResolve	Global hereditary	H-disjoint families, LT-family audit, strong sidecar.
BUD	Budget resolver	Neutral grammar, envelope DP, active routing, strong sidecar.
RW/FF	Residual witnesses	FF-LOSS, MuBridge, SpanPolicy, SaturatePositive, BCELReady.
BN2-BN6	Cellization	Side-tight bases, request envelopes, cancellation, shadows, hypergraphs.
PkgC	Separating consumers	Request traces, full restoration universes, matching graphs.
Packet	Selector seeds	Pair, balanced-triple, and full-span spine extraction.
R	Realizers	Gain-only public interface, typed $\perp$, charge surplus.
HB	Rank negatives	HN-BUD blocker graph, selector-silence induction.
O	Oracle	H/B negatives, selector silence, ZeroSlack, residual descent.
G	SAT embedding	Macros, locked NAND threshold, residual slack at most four.
PACK	Top-level checker	Interface, row keys, imports, reflection, bounds, no hidden minimization.

26 Closing summary

The programme reduces SAT to exact minimization of a locked multi-output NAND direct-wire word with residual slack at most four, and proves that residual-band exact minimization is polynomial when the finite proof-carrying package accepts. The structural proof supplies compatible support replacement, charge soundness, mode projection control, finite routers, hereditary and budget closure, packet extraction, selector realization, ZeroSlack, locked NAND thresholding, and generated package sufficiency.

The final repository artefact stack discharges the generated-package condition. The accepted proof report records accepted `CheckPCCPackexp0`, accepted `CheckAcceptRun0`, accepted replay, accepted final verdict, accepted final certificate, accepted release audit, accepted final release gate, and accepted `CheckFinalPNPProofReport0`. The final public theorem statement is

$$\mathrm{CheckPCCPack}_{\mathrm{exp}}(\mathrm{GeneratePCCPack}()) = \mathsf{accept} \quad \Longrightarrow \quad P = NP,$$

and the sealed artefact stack records the antecedent as accepted. Hence the final conclusion of this paper is

$$P = NP.$$